Praxis der Softwareentwicklung

G-Cal

Wintersemester 2025/26

Entwurf

Karlsruher Institut für Technologie (KIT)

Karlsruhe, den 09.01.2026

Karlsruher Institut für Technologie
Fakultät für Informatik
Postfach 6980
76128 Karlsruhe

Inhaltsverzeichnis

1 Ergänzung Pflichtenheft 6
1.1 Lokale Daten . . . 6
1.2 Produktfunktionen . . . 6
1.2.1 Xp-vergabe . . . 6
1.3 Benutzeroberfläche (GUI) . . . 9

2 Architektur 11
2.1 Struktureller Aufbau . . . 11
2.1.1 Systemkomponenten . . . 11
2.1.2 Externe Komponenten . . . 12
2.1.3 Verteilung . . . 12
2.2 Architektur . . . 12
2.2.1 Client-Server Architektur . . . 13
2.2.2 Client Struktur . . . 13
2.2.3 Client Entwurfsentscheidungen . . . 14
2.2.4 Server Struktur . . . 14
2.2.5 Server Entwurfsentscheidungen . . . 14
2.2.6 Datenbank . . . 15
2.3 Schnittstellenspezifikation . . . 15
2.3.1 SQL Schema . . . 15
2.3.2 REST API . . . 17
2.3.2.1 API Entwurfsentscheidungen . . . 17
2.3.2.2 API Endpunkte . . . 17
2.3.3 Client-interne Schnittstellen . . . 18
2.3.3.1 Model Facade . . . 18
2.3.3.2 RequestAPI . . . 18
2.3.4 Server-interne Schnittstellen . . . 18
2.4 Externe Bibliotheken . . . 19
2.4.1 Jetpack Compose . . . 19
2.4.2 Ktorm . . . 19
2.4.3 Ktor . . . 19
2.4.4 kotlinx.serialization . . . 20

3 Feinentwurf **21**
3.1 Systementwurf 21
3.1.1 Data Transfer Objects (DTOs) 21
3.1.2 Authentifizierungsablauf 21
3.2 Server Entwurf 24
3.2.1 Datentypen 24
3.2.2 Serverschnittstellen 25
3.2.2.1 Controller 25
3.2.2.2 RequestHandler 25
3.2.2.3 Authorisierung 27
3.2.2.4 Eventverwaltung 27
3.2.2.5 Gruppenverwaltung 27
3.2.3 Controller 28
3.2.4 RequestHandler 28
3.2.5 Eventverwaltung 28
3.2.6 Gruppenverwaltung 28
3.2.7 Authentifizierung 29
3.2.8 Nutzerverwaltung 29
3.2.9 Datenbankzugriff 30
3.2.10 Diagramme 31
3.3 Client Entwurf 35
3.3.1 Userinterface 35
3.3.1.1 Login Screen (Realisierung von F20) 35
3.3.1.2 Main Screen: Kalender-Hub & Event-Management 41
3.3.1.3 RanglisteScreen - (Realisierung von F160) 52
3.3.1.4 BerichteScreen (Realisierung von F140) 58
3.3.1.5 ProfileScreen (Realisierung von F150, F30, F40) 65
3.3.2 ViewModel-Schicht 76
3.3.2.1 View und ViewModel Umsetzung 76
3.3.2.2 View-ViewModel Schnittstelle (Unidirectional Data Flow) 76
3.3.2.3 LoginViewModel 77
3.3.2.4 LeaderboardViewModel 79
3.3.2.5 ReportViewModel 79
3.3.2.6 ProfileViewModel 80
3.3.2.7 MainViewModel (Kalender & Editor) 81
3.3.3 Model 84

3.3.3.1 Model Facade 84
3.3.3.2 RequestAPI 86
3.3.3.3 Interfaces 88
3.3.3.4 Implementierung 91
3.3.3.5 Weitere Klassen 95
3.3.3.6 Lokale Datenspeicherung 99

A SQL-Schemadefinition 103

Abbildungsverzeichnis

1.1 Darstellung der Belohnungs-XP-Werte 9

2.1 Systemkomponenten 11
2.2 Verteilungsdiagrmam 12
2.3 Schemadefinition der Datenbank 16

3.1 Aktivitätsdiagramm für Login/SignUp-Bildschirm 22
3.2 Aktivitätsdiagramm für Profilbearbeitungsbildschirm 23
3.3 Beschreibung des Controller-Layers 31
3.4 Klassen zur Event- und Gruppenverwaltung 32
3.5 Die Datentypen des Servers 33
3.6 Klassen zur Nutzerverwaltung 34
3.7 Schnittstelle zur Datenbankanbindung 34
3.8 Zustandsdiagramm: Innere Logik des LoginViewModels. 78
3.9 Sequenzdiagramm: Ablauf beim Erledigen eines Events. Zeigt den asynchronen Aufruf an die ModelFacade. 82
3.10 Klassendiagramm der ViewModel-Schicht mit zentraler ModelFacade. Alle ViewModels nutzen die Facade für Datenzugriffe. . . . 83
3.11 Klassendiagramm zeigt die ModelFacade und RequestAPI Schnittstellen und deren Implementierungen. 93
3.12 Klassendiagramm beshreibt alle Implementationen des User Interfaces 93
3.13 Klassendiagramm beschreibt die Vererbungen und Implementationen des Event interfaces. 94
3.14 Klassendiagramm zeigt die Zwei implementationen von CompletionDetail. Identifier ist im fall von Events die Eventid, und für achievements der name. 94
3.15 Klassendiagramm der ModelData Klasse und ihren Observern. . 96
3.16 Klassendiagramm der Lokalen Datenbank. Zeigt welche Dao mit welchen Entitys Interagieren. Getter und settermethoden der Entitys wurden weggelassen. 102

1 Ergänzung Pflichtenheft

1.1 Lokale Daten

Aufgrund der umsetzung der Autentifizierung mit Open Id Connect ist das speichern des Passworts auf dem Server unnötig. Unsere Serverdaten sind also Aktualisiert nurnoch:

- Benutzername
- XP-Wert
- Events
- Gruppen

Ein weiterer Zusatz sind ToDo's, welche sich von Terminen unterscheiden. In Pflichtenhgeft werden in diesem Abschnitt Termine als Events bezeichnet, was eingentlich der überbegrif für ToDo und Termine ist. ToDo's speichern folgende Daten:

- ToDo Name
- Deadline
- Gruppe

Dabei ist Deadline ein Datum, nachdem das ToDo nichtmehr erledigt werden kann. Deadline sowie Gruppe sind jedoch optional, so wie für die Gruppen bei Terminen soll es hier für Gruppen und Deadline Platzhalter geben. Es gibt somit drei verschiedene Arten von Events: Termine, Shared Events und ToDo's. Termine sind Events die eine feste Zeit haben, wo sie beginnen und eine feste Zeit, wo sie enden. Shared Events sind Termine, an denen mehrere Benutzer teilnehmen. ToDo's haben keine feste Anfangszeit, aber können eine Endzeit (Deadline) haben.

1.2 Produktfunktionen

1.2.1 Xp-vergabe

Für welche Termine wird Xp-Vergeben?
Der Xp-Wert wird in der Regel am vortag verteilt. Ist das nicht geschehen,

also kein Event hat einen Xp wert am aktuellen Tag wenn die App das erstemal startet, wird für diesen Tag Xp verteilt. Dadurch werden nachträglich hinzugefügten events kein xp wert mehr zugeteilt.

Wie wird Xp an Termine verteilt?
Es wird immer ein ganzer Tag betrachtet und es gibt einen Tages-höchstwert, das bedeutet die Summe an Xp aller Termine eines Tages muss kleiner als dieser Tageshöchstwert sein. Der Xp-wert soll für den Tag betrachtet ungefär Proportional zur Dauer des Termins sein, dabei ist ein gewisse ungenauigkeit erwünscht.

Xp-Vergabe an Termine des Folgetages

Anforderung

Ziel Zuordnung des Xp-Werts, der für das Abschließen des zugehörigen Termins vergeben wird, für die Termine des folgenden Tages.

Vorbedingung App ist installiert; Für den nächsten Tag wurden noch nicht für alle Termine Xp-Werte bestimmt; Der nächste Tag besitzt mindestens einen Termin.

Nachbedingung Erfolg jeder Termin des kommenden Tages hat einen zugeteilten Xp-Wert. Die Gesammtsumme an Xp des Folgetages überschreitet nicht die Obergrenze.

Nachbedingung Fehlschlag

Akteure System

Auslösendes Ereignis Hinzufügen eines neuen Termins; Erstes Öffnen der App an einem Tag;

Beschreibung Der Client prüft nach dem hinzufügen eines Termins, oder beim ersten öffnen der App an einem Tag, den folgenden Tag. Wenn an diesem Tag Termine noch keinen XP-wert zugeteilt haben, werdem jedem Termin neue Xp-werte Zugeteilt.

Xp-Vergabe an Termine von Heute

Anforderung

Ziel Zuordnung des Xp-Werts, der für das Abschließen des zugehörigen Termins vergeben wird, für die Termine des heutigen Tages.

Vorbedingung App ist installiert; Für den heutigen Tag wurden für keine Termine Xp-Werte bestimmt; Der heutige Tag besitzt mindestens einen Termin.

Nachbedingung Erfolg jeder Termin des heutigen Tages hat einen zugeteilten Xp-Wert. Die Gesammtsumme an Xp dvon heute überschreitet nicht die Obergrenze.

Nachbedingung Fehlschlag

Akteure System

Auslösendes Ereignis Erstes Öffnen der App an einem Tag;

Beschreibung Der Client prüft beim ersten öffnen der App an einem Tag, den aktuellen Tag. Wenn an diesem Tag alle Termine noch keinen XP-wert zugeteilt haben(und Termine existieren), wird jedem Termin ein Xp-werte Zugeteilt.

Wie wird Xp an ToDo verteilt?
Xp für ToDos werden erst bestimmt, wenn das ToDo abgeschlossen wird. Der Xp-Wert soll Zufällig einen von vier Werten annehmen, die jedoch unterschiedlich wahrscheinlich sind. Diese vier Werte selbst sollen proportional zu den, für den heutigen Tag durch Termine gesammelten Xp sein.

Xp-Vergabe an ToDo

Anforderung

Ziel Zuordnung des XP-Werts, der für das Abschließen des ToDo's vergeben wird.

Vorbedingung App ist installiert; Es gibt mindestens ein unerledigtes ToDo

Nachbedingung Erfolg Es wird ein Xp-wert bestimmt

Nachbedingung Fehlschlag

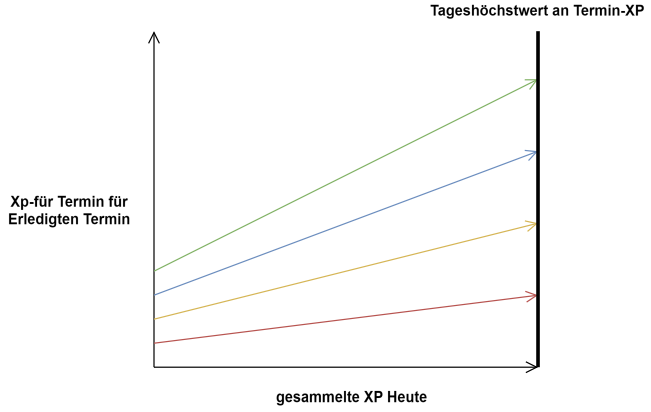


Abbildung 1.1: Skizziert den Verlauf der Verschieden möglichen XP-Werten

Akteure System

Auslösendes Ereignis Abschließen eines ToDo

Beschreibung Der Client berechnet beim Abschließen eines Todos den aktuellen wert der vier möglichen Xp-Werte, wählt zufällig einen aus und teilt die Xp-dem ToDo zu (Dieses ist direkt danach auch abgeschlossen).

1.3 Benutzeroberfläche (GUI)

Im Vergleich zu den ursprünglichen Entwürfen und Skizzen im Pflichtenheft wurden signifikante Anpassungen an der Benutzeroberfläche vorgenommen, um die Usability zu verbessern und technische Architekturentscheidungen widerzuspiegeln.

Wegfall von Authentifizierungs-Screens

Da die Authentifizierung nun vollständig über **OpenID Connect** (Google) abgewickelt wird, entfallen die dedizierten Screen für *Registrierung*. Infolgedessen sind auch die ursprünglich geplanten Masken zur Änderung von Benutzerdaten (wie Passwort ändern oder Vor-/Nachname bearbeiten) obsolet, da diese Datenverwaltung nun extern beim Identity Provider liegt.

Optimierung der Navigation (BottomBar)

Die globale Navigationsleiste (`BottomNavigationBar`) wurde entschlackt, um die Benutzerergonomie zu steigern:

- **Event-Editor:** Der Editor ist kein eigener Tab mehr in der BottomBar. Stattdessen wurde er als **Floating Action Button (FAB)** mit einem +-Symbol direkt in den `MainScreen` integriert. Dies ermöglicht eine schnellere Erstellung von Einträgen ohne Kontextwechsel und verbessert den Navigationsfluss.
- **Einstellungen (Settings):** Auch der Settings-Bereich wurde aus der BottomBar entfernt. Die Einstellungen sind nun als Sub-Screen über den `ProfileScreen` erreichbar. Diese Hierarchieanpassung bündelt konto-relevante Aktionen an einem Ort und reduziert die Komplexität der Hauptnavigation.

2 Architektur

2.1 Struktureller Aufbau

G-Cal wird in drei Systemkomponenten aufgeteilt, eine **Android App**, einen **Server** sowie eine **SQL Datenbank**. Die Kommunikation zwischen App und Server erfolgt über eine REST Schnittstelle, die Datenbankabfragen werden mittels SQL gestellt. Die Android App sowie der Server werden in Kotlin entwickelt. Im folgenden Abschnitt wird die genaue Spezifikation des Systems beschrieben.

2.1.1 Systemkomponenten

Android App (kurz "Client")
Die Android App kommuniziert über die GUI mit dem Endnutzer. Dabei ist sie verantwortlich für die Verarbeitung der Nutzereingaben sowie der Serverantworten. Für die erfolgreiche Kommunikation zwischen App und Server mittels Rest API muss eine HTTPS-Verbindung aufgebaut werden.

Server
Der Server regelt die Kommunikation zwischen den Nutzern durch bspw. Shared Events. Außerdem verarbeitet er Systemänderungen, indem er prüft, ob diese valide sind und im erfolgreichen Fall die Änderungen in der Datenbank persistiert.

SQL Datenbank
Die SQL Datenbank speichert alle systemrelevanten Nutzerdaten wie Events, Gruppen und persönliche Informationen, um diese dem Server auf Anfrage zur Verfügung zu stellen.

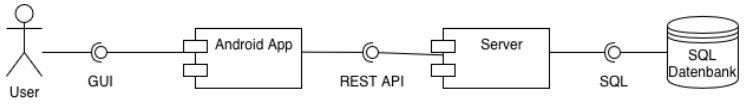


Abbildung 2.1: Darstellung der einzelnen Systemkomponenten

2.1.2 Externe Komponenten

OpenIP Connect Identity Provider

Der Identity Provider von Google prüft die Authentizität eines Nutzers und liefert beim Login des Nutzers den benötigten Authentification Code.

2.1.3 Verteilung

Wie zuvor beschrieben besteht *G-Cal* aus vier Hauptkomponenten. Da der Open-ID Connect Identity Provider von Google eine externe Komponente ist, wird er nicht für die Verteilung berücksichtigt.

Die restlichen Komponenten lassen sich in *Server* und *Android Phone* unterteilen.

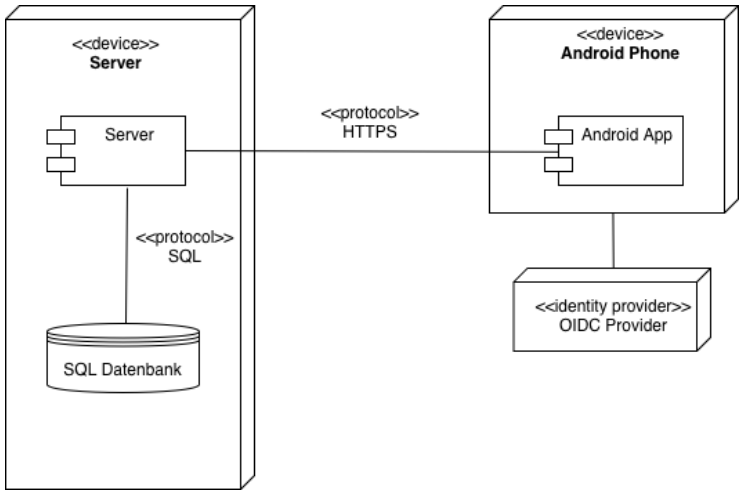


Abbildung 2.2: Unterteilung der Systemkomponenten im Verteilungsdiagramm

2.2 Architektur

Der Folgende Abschnitt beschreibt die Architektur des Systems auf abstrakter Ebene.

2.2.1 Client-Server Architektur

Die Anwendung G-Cal wird durch eine Client-Server Architektur modelliert. Diese Modellierung ist erforderlich, da die Termine pesistent in einer Datenbank gespeichert werden müssen, damit ein Nutzer von verschiedenen Geräten mit seinem Account darauf zugreifen kann. Die Speicherung der Nutzerdaten wie Events übernimmt der Server in einer Datenbank. Die Trennung von Client und Server ermöglicht außerdem eine Authentifizierung des Nutzers sowie eine bessere Wartbarkeit der App.
Um zusätzlich im Offline-Betrieb dem Nutzer die Events anzuzeigen, speichert der Client die Event-Daten zusätzlich lokal in einer Room-Datenbank. Damit ist eine eingeschränkte Nutzung der App auch ohne Internet-Verbindung möglich.

2.2.2 Client Struktur

Der Client nutzt das MVVM-Architekturmuster und ist entsprechend in die drei Komponenten **View**, **Model** und **ViewModel** aufgeteilt. Die Kommunikation zwischen den Schichten erfolgt ausschließlich über das **ViewModel**, d.h. **Model** und **View** kommunizieren nicht direkt miteinander.

View
: Verantwortlich für die Darstellung der UI und zeigt den UI Zustand an, der sich aus den Informationen des ViewModels ergibt.

Model
: Modelliert die zugrundeliegende Logik des Clients und verwaltet die Kommunikation des Clients mit dem Server

ViewModel
: Sorgt für die Kommunikation zwischen View und Model, in dem es die Nutzereingaben vom View sowie die Informationen des Models entsprechend verarbeitet und weiterleitet.

2.2.3 Client Entwurfsentscheidungen

Die Unterteilung des Clients in ein Schichtenmodel, bestehend aus Model, View und ViewModel, aufzuteilen hat verschiedene Vorteile. Durch die Abstraktion des Zugriffs auf die Server API durch das ViewModel können UI und Logik implementiert werden, ohne die genaue Implementierung des Servers zu kennen. Auch die Teilung von UI und Logik in View und Model entspricht diesem Prinzip, es ermöglicht eine geteilte Entwicklung der beiden Komponenten, wobei die Kommunikation der drei Komponenten über Schnittstellen geregelt wird.

2.2.4 Server Struktur

Der Server wird in drei intransparente Schichten gegliedert, **Controller**, **Model** und **Persistence**.

Oberste Schicht: Controller
: Die oberste Schicht kommuniziert direkt mit dem Client und verwaltet demnach die Verarbeitung von API Calls. Dabei übersetzt der Controller die Calls in die zuständigen Funktionen des Models, die die weitere Verarbeitung der Calls umsetzten. Damit ist der Controller auch dafür zuständig, dem Client Informationen zu übermitteln.

Mittlere Schicht: Model
: Das Model ist für die zentrale Logik des Servers zuständig. Dazu verarbeitet er die Anfragen des Controllers und der Persistence entsprechend.

Unterste Schicht: Persistence
: Die Persistence ist für die direkte Kommunikation mit der Datenbank mittels Ktorm zuständig und erhält demnach Anfragen vom Model und übersetzt diese in SQL Inserts und Queries.

2.2.5 Server Entwurfsentscheidungen

Die Nutzung der Schichten-Architektur erlaubt es eine Trennung der verschiedenen Funktionalitäten, die der Server übernehmen soll. Dadurch wird die Wartbarkeit sowie Performance des Servers erhöht. Für die Datenbankabstraktion wird *Ktorm* gewählt, dabei haben wir uns bewusst für die ORM-Funktionalität

entschieden, da diese eine Abtraktion für die häufig gestellten SQL-Queries direkt in Kotlin bietet.

2.2.6 Datenbank

G-Cal nutzt PostgreSQL als Datenbankmanagementsystem. Relationale Datenbanken erlauben einen einfachen Zugriff auf die gespeicherten Daten mittels SQL und die genutzte Bibliothek *Ktorm* vereinfacht den Zugriff auf diese direkt in Kotlin. Grunde für die Wahl des DBMS sind Stabilität und Zuverlässigkeit sowie die Unterstützung in Kotlin.

2.3 Schnittstellenspezifikation

2.3.1 SQL Schema

Die folgende Grafik zeigt, wie die Daten der Nutzer in der Datenbank des Servers gespeichert werden. Damit werden die Speicher-Anforderungen des Pflichtenheftes erfüllt. Die Daten für die Accounts mit Benutzername, Anzeigename und Erfahrungspunkte werden persistent gespeichert. Außerdem werden die Events und Gruppen der einzelnen Nutzer gespeichert, dazu werden die Beziehungen modelliert.

Die SQL Befehle zum Anlegen des Relationenschemas befinden sich im Anhang.

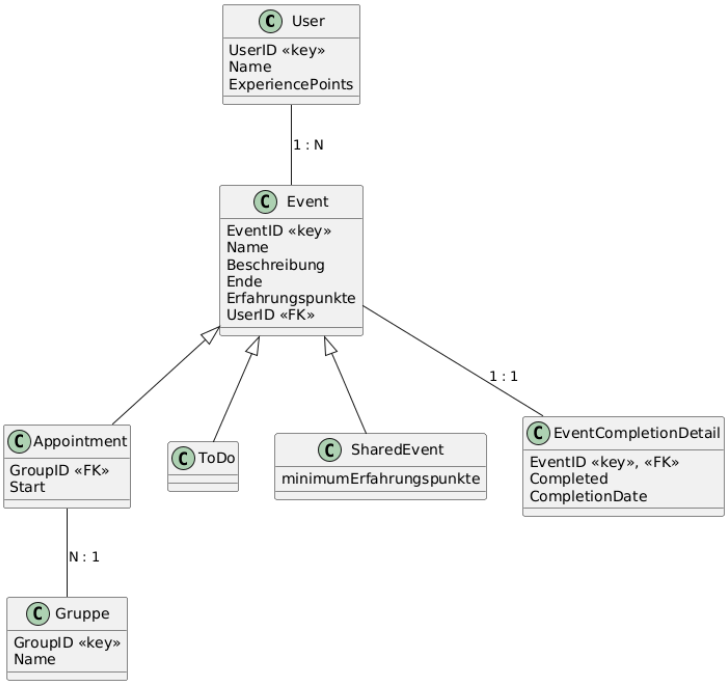


Abbildung 2.3: Schemadefinition der Datenbank

2.3.2 REST API

2.3.2.1 API Entwurfsentscheidungen

Methode GET

Wir verwenden GET um Datenabfragen durchzuführen (read only).

Methode PUT

Wir verwenden PUT um bereits bestehende Werte zu aktualisieren.

2.3.2.2 API Endpunkte

/login

Dieser Endpunkt wird für das Autentifizieren der Benutzer verwendet.

/user

Diese Route verwendet der Client um Abfragen über sein eigenen Account zu machen

/user/experiencePoints

Unter dieser Route finet sich der Xp-wert des Benutzers.

/user/groups

Unter dieser Route befinden sich alle Gruppen, welche der nutzer erstellt und nicht gelöscht hat.

/user/events

Unter dieser Route befinden sich alle Events, welche der nutzer erstellt hat/beigetreten ist und nicht gelöscht hat.

/user/events/termine

Unter dieser Route befinden sich alle Termine, welche der nutzer erstellt und nicht gelöscht hat.

/user/events/toDo

Unter dieser Route befinden sich alle ToDo, welche der nutzer erstellt und nicht gelöscht hat.

/user/events/sharedEvents

Unter dieser Route befinden sich alle shared Events, welche der nutzer abgeschlossen hat oder beitreten kann/beigetreten ist.

/user/friends
Unter dieser Route befinden sich alle accounts, welche der nutzer als Freunde hinzugefügt hat.

/users
Unter diesem Endpunkt können die Öffentlichen Daten anderer Nutzer eingesehen werden.

2.3.3 Client-interne Schnittstellen

2.3.3.1 Model Facade Das Model Stellt dem Viewmodel die Einheitliche Schnitstelle ModelFacade bereit. Das Interface bietet Methoden um die Meisten ANfragen des Viewmodels zu beantworten, bzw die nötigen Objekte liefert um die Anfragen zu beantworten.

2.3.3.2 RequestAPI Das Model Kommuniziert mit der REST API des Servers über die RequestAPI Schnitstelle. Das Interface bietet für jeden Endpunkt, bis auf Login eine methode und wrappt die Http Responses des Servers in die Zugehörigen Datentypen.

2.3.4 Server-interne Schnittstellen

Der folgende Abschnitt beschreibt die einzelnen Schnittstellen des Servers.

Authentifizierung Die Authentifizierungschnittstelle prüft mittels eines Tokens, ob eine Nutzeranfrage tatsächlich von dem gegebenen Nutzer stammt.

Authorisierung Die Schnittstelle prüft, ob ein Nutzer Zugriff auf ein Event bzw. eine Gruppe mit einer bestimmten *EventID* oder *GroupID* besitzt. Damit wird sichergestellt, dass ein Nutzer nicht Daten anderer Nutzer modifizieren oder löschen kann.

Nutzeradministration Die Nutzeradministration ermöglich das Hinzufügen, Löschen und Abfragen von Accounts.

Eventverwaltung Die Eventverwaltungs-Schnittstelle erlaubt es, Events zu erstellen, bearbeiten und löschen.

Gruppenverwaltung Diese Schnittstelle ermöglicht das Erstellen und Löschen von Gruppen.

Datenbankzugriff Der Datenbankzugriff erfolgt mittels ORM-Funktionaliät von Ktorm.

2.4 Externe Bibliotheken

2.4.1 Jetpack Compose

Für die Entwicklung der graphischen Benutzeroberfläche wird *Jetpack Compose*[1] benutzt. Der Vorteil dieser Bibliothek gegenüber anderen UI-Bibliotheken für Android ist die intuitive Verwendung der deklarativen API. Dadurch wird die Entwicklung durch weniger Code und einfachere Anpassung verbessert. Jetpack Compose ist auch die von Android empfohlene Bibliothek.

2.4.2 Ktorm

Die Datenbankzugriffe des Servers werden mittels *Ktorm*[2] implementiert. Die ORM-Funktionalität erlaubt dabei eine Abbildung der Datenbankobjekte auf Programmierobjekte, was den Zugriff auf die Datenbank erleichtert. Ktorm erlaubt außerdem die Datenbankdefinition sowie die Formulierung von Datenbankanfragen innerhalb von Kotlin.

2.4.3 Ktor

Für die verschlüsselte Kommunikation zwischen Client und Server mittels HTTPS wird *Ktor*[3] verwendet. Da die Kommunikation potentiell sensible Daten enthält, ist die Verschlüsselung dieser essentiell. Ktor lässt sich nahtlos in Kotlin integrieren und erlaubt asynchrone Kommunikation zwischen Client und Server.

[1] https://developer.android.com/compose
[2] https://www.ktorm.org/
[3] https://ktor.io/

2.4.4 kotlinx.serialization

Zur einfachen Serialisierung bzw. Deserialisierung von Kotlin-Objekten im JSON-Format für die Übertragung mit HTTP wird die Bibliothek *kotlinx.serialization*[4] verwendet.

[4] https://github.com/Kotlin/kotlinx.serialization

3 Feinentwurf

Das folgende Kapitel beschreibt im Detail, wie die in Kapitel 2 beschriebene Architektur umgesetzt werden soll.

3.1 Systementwurf

3.1.1 Data Transfer Objects (DTOs)

Für den einheitlichen Datentransport zwischen Client und Server werden *Data Transfer Objects* verwendet. Dazu wird die in Kapitel 2 beschriebene kotlinx.serialization Bibliothek verwendet. Diese erlaubt die Serialisierung von DTOs in JSON-Format, welche dann im HTTP Body zwischen Client und Server versendet werden können. Der Empfänger kann die empfangenen Daten dann mit der gleichen Bibliothek wieder deserialisieren.

3.1.2 Authentifizierungsablauf

Bei Registrierung/Anmeldung mit Benutzerdaten erhält der Client einen Session-Token vom Server. Der Server fragt diesen nur bei Anmeldungen vom Client ab und prüft ihn auf Gültigkeit (Ablaufdatum), und falls er noch gültig ist, ist der Nutzer direkt angemeldet. Bei Abmeldung wird der Token sofort invalidiert.

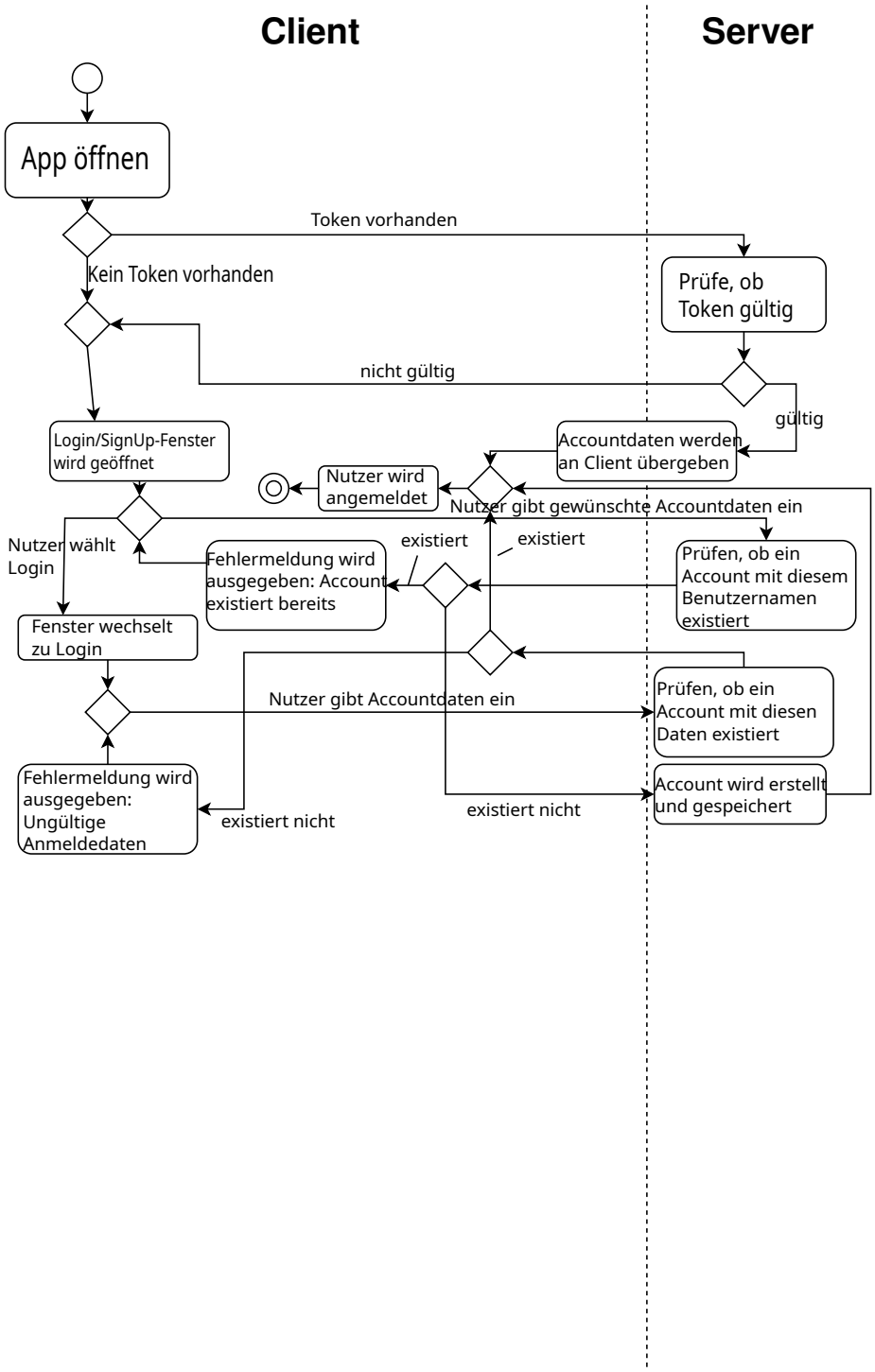


Abbildung 3.1: Beschreibt den Ablauf von Anmeldung und Registrierung.

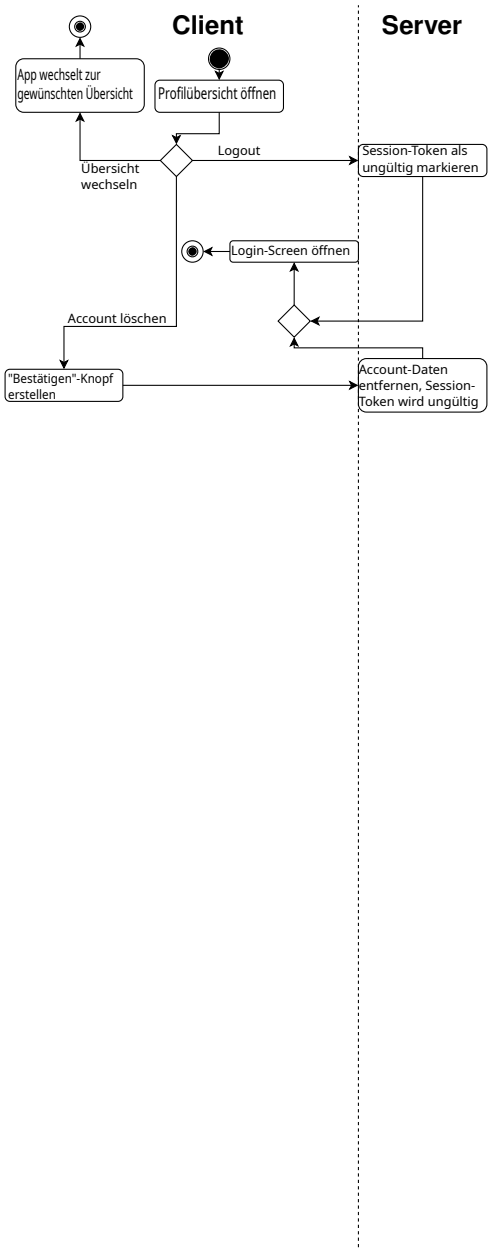


Abbildung 3.2: Beschreibt den Ablauf der Abmeldung und der Löschung des Accounts

3.2 Server Entwurf

Im Folgenden wird die Implementierung des Servers im Detail erläutert.

3.2.1 Datentypen

Alle im folgenden erläuterten Datentypen sind unveränderlich (immutable).

Um in Methodenaufrufen zu verifizieren, dass übergebene Parameter tatächlich valide IDs sind, benutzen wir Wrapper-Klassen, die den Typ sowie Format der enthaltenen Strings prüfen. Damit wird Typsicherheit garantiert. Diese Klassen sind *Username*, *EventID* und *GroupID*. Alle zusätzlichen Informationen werden dann in den Klassen User, Calendar, Event und Group gespeichert.

Außerdem gibt es den generischen Datentyp *Result* $\langle T \rangle$. Da Kotlin nur unchecked Exceptions besitzt, ist es bewährte Praxis, eine Klasse zu implementieren, die entweder ein Ergebnis vom Typ T oder eine Fehlermeldung besitzt.

Die Events der Nutzer werden in der Klasse Events modelliert und in der Datenbank persistiert. Dazu gehört die Unterteilung von Events in Termine, ToDos und SharedEvents. Desweiteren werden die Gruppen, die ein Nutzer erstellen kann, in der Klasse Group dargestellt. Diese enthält eine Liste an zugehörigen Events. Gruppen und Events werden respektiv durch *GroupID* und *EventID* identifiziert, dabei ist jede Gruppe und jedes Event dem zugehörigen Nutzer zugeordnet.

Spezifikationen wie die Zugangsdaten für die Datenbank oder Port und URI vom Server werden in einem Config-File dem Server übergeben, der bei Start des HTTPS-Servers die Werte der Config Datei einliest und in einem Config-Objekt speichert.

3.2.2 Serverschnittstellen

Um die konkrete Implementierung der einzelnen Handlers zu abstrahieren verwenden wir Schnittstellen, die Flexibilität und Testbarkeit bieten.

3.2.2.1 Controller

Die Schnittstelle des Controllers erlaubt das Starten und Stoppen des HTTPS-Servers, der über *Ktor* läuft.

run() : Result $\langle Unit \rangle$
: Startet den HTTPS-Server.

stop() : Result $\langle Unit \rangle$
: Stoppt den laufenden HTTPS-Server.

3.2.2.2 RequestHandler

Die RequestHandler Schnittstelle gehört zur Controller-Schicht und bietet für jede REST-API Route, abgesehen vom Login, einen Funktionsaufruf an.

getUserInfo(Username): Result $\langle String \rangle$
: Liefert die Nutzerinformationen zu einem bestimmten Benutzernamen, also Anzeigename und Erfahrungspunkte.

increaseExperiencePoints(Username, Points) : Result $\langle Unit \rangle$
: Erhöht die Erfahrungspunkte eines Nutzers um einen bestimmten Betrag

getExperiencePoints(Username): Result $\langle Integer \rangle$
: Gibt das XP-Levels eines gegebenen Nutzers zurück

createGroup(Username, Groupname) : Result $\langle Unit \rangle$
: Die Methode erstellt für einen gegebenen Nutzer eine neue Gruppe mit gegebenem Namen.

getGroups(Username): Result $\langle List\langle Group \rangle \rangle$
: Gibt die eine Liste an *Group* zurück, die zu einem Nutzer gehören.

createAppointment(Username, AppointmentData) : Result $\langle Unit \rangle$
: Erstellt ein neuen Termin für einen gegebenen Nutzer.

createToDo(Username, ToDoData) : Result $\langle Unit \rangle$
Erstellt eine neue ToDo für einen gegebenen Nutzer.

createSharedEvent(Username, SharedEventData) : Result $\langle Unit \rangle$
Erstellt ein neues SharedEvent für einen gegebenen Nutzer.

modifyEvent(Username, modifiedEventData) : Result $\langle Unit \rangle$
Bearbeitet ein bestehendes Event für einen gegebenen Nutzer, in dem ein bereits existierendes Event durch das neue *modifiedEvent* ersetzt.

deleteEvent(Username, EventID) : Result $\langle Unit \rangle$
Löscht ein bestehendes Event mit gegebener EventID für einen Nutzer.

getEvents(Username) : Result $\langle List\langle Event \rangle \rangle$
Gibt eine Liste bestehendes aus allen *Event* eines Nutzers zurück.

addFriend(Username, Username): Result $\langle Unit \rangle$
Fügt für einen Nutzer einen anderen Nutzer als Freund hinzu, sobald der andere Nutzer dies akzeptiert.

removeFriend(Username, Username) : Result $\langle Unit \rangle$
Löscht einen gegebenen Nutzer als Freund.

getFriends(Username) : Result $\langle List\langle Username \rangle \rangle$
Gibt für einen gegebenen Nutzer eine Liste an *Username* von allen seinen Freunden zurück

users() : Result $\langle Map\langle Username, String \rangle \rangle$
Gibt eine Map zurück, die alle Nutzer auf zugehörige Nutzerinformationen abbildet.

createGlobalChallenge() : Result $\langle Unit \rangle$
Erstellt eine neue globale Challenge, wenn diese vom Server getriggert wird.

3.2.2.3 Authorisierung

Die Authorizer Schnittstelle bietet die Möglichkeit, zu verifizieren, dass ein Nutzer mit einem gegebenen Nutzernamen Zugriff auf ein bestimmtes Event oder auf eine Gruppe mit einer *EventID* bzw. *GroupID* hat.

authorizeEvent(Username, EventID) : Result $\langle Boolean \rangle$

Prüft, ob ein Nutzer mit gegebenem Nutzernamen Zugriff auf ein Event mit gegebener *EventID* hat und gibt, falls es eine valide Anfrage war, ein Result mit True oder False zurück.

authorizeGroup(Username, GroupID) : Result $\langle Boolean \rangle$

Prüft, ob ein Nutzer mit gegebenem Nutzernamen Zugriff auf eine Gruppe mit gegebener *GroupID* hat und gibt, falls es eine valide Anfrage war, ein Result mit True oder False zurück.

3.2.2.4 Eventverwaltung

Das Interface EventManagement verwaltet die Verarbeitung von Events. Dazu stellt es die Methoden bereit, die über den RequestHandler aufgerufen werden.

createEvent(Username, Event) : Result $\langle Unit \rangle$

Die Methode erstellt ein neues Event für einen Nutzer und speichert dieses in der Datenbank.

modifyEvent(Username, modifiedEvent) : Result $\langle Unit \rangle$

Die Methode verändert ein bereits bestehendes Event und ersetzt dieses durch das neue *modifiedEvent*.

deleteEvent(Username, EventID) : Result $\langle Unit \rangle$

Der Aufruf dieser Methode löscht das Event mit der gegebenen EventID aus der Datenbank, falls es zu dem gegebenen Nutzer gehört.

3.2.2.5 Gruppenverwaltung

Das Interface GroupManagement ist verantwortlich für das Erstellen und Löschen von Gruppen für einen Nutzer.

createGroup(Username, Groupname) : Result $\langle Unit \rangle$

Erstellt für einen Nutzer eine neue Gruppe mit gegebenen Gruppennamen

deleteGroup(Username, GroupID) : Result $\langle Unit \rangle$

Löscht eine Gruppe mit gegebener *GroupID*, falls diese exisitert. Falls die Gruppe nicht exisitert, gibt es eine Fehlermeldung.

3.2.3 Controller

Die Implementierung der Controller-Schnittstelle wird durch den RequestController bereitgestellt. Dieser wird über die Methode *create()* erzeugt und erhält das Config-Objekt, den Authenticator, den Authorizer sowie den RequestHandler.

Der RequestController startet und stoppt den HTTPS-Server mittels Ktor. Erhält der RequestController über die vordefinierten Routen eine Request, prüft zunächst durch den Authenticator und Authorizer, ob die Request des Nutzers valide ist. Ist die Request valide, übersetzt der RequestController die Route in einen Funktionsaufruf des RequestHandlers. Für den Fall, dass die angegebene Route der Login ist, wird die passende Methode des Authentificators aufgerufen.

3.2.4 RequestHandler

Zur Umsetzung der RequestHandler-Schnittstelle wird die RequestDispatcher Klasse genutzt. Diese besteht aus den beiden Klassen UserDispatcher und EventDispatcher. Da sich die REST Routen in diese beiden Kategorien unterteilen lassen, leitet der RequestDispatcher Requests passend weiter, bei einer Abfrage bezüglich Nutzer wird der UserDispatcher genutzt, behandelt die Abfrage Events, so wird der EventDispatcher genutzt.

3.2.5 Eventverwaltung

Die Schnittstelle EventManagement wird durch die Klasse EventManager implementiert. Dabei verwaltet der EventManager zusätzlich einen ResourceAuthorizer, welcher die Schnittstelle Authorizer umsetzt. Dies ermöglicht es zu überprüfen, ob der Zugriff eines Nutzers auf ein Event gültig ist.

3.2.6 Gruppenverwaltung

Die Klasse GroupManager implementiert die Schnittstelle GroupManagement. Ähnlich wie der EventManager verwaltet er einen ResourceAuthorizer, welcher

wiederum die Schnittstelle Authorizer implementiert. Dadurch wird sichergestellt, dass ein Nutzer für Zugriffsrechte auf die Gruppe hat.

3.2.7 Authentifizierung

Zur Authentifizierung des Tokens eines Nutzers wird die Klasse UserAuthenticator benutzt. Diese bietet die Methode authenticate an, welche über die API-Route /login aufgerufen wird. Diese erhält ein Token und prüft, ob dieser valide ist. Ist der Nutzer noch nicht intern registriert, so wird ein neuer Account angelegt.

authenticate(Token) : Result $\langle Account \rangle$

Nimmt ein Token vom OpenID Connect Identity Provider und prüft, ob dieses valide ist. Ist es valide, prüft die Methode, ob der Nutzer bereits einen internen Account hat. Ist dies der Fall, wird der Account zurückgegeben, sonst wird ein neuer Account erstellt und dieser zurückgegeben. Da der wrapped Type Result verwendet wird, kann im Fall eines invaliden Tokens eine Fehlermeldung zurückgegeben werden.

3.2.8 Nutzerverwaltung

Die Klasse UserManager verwaltet die Accounts der Nutzer und stellt die Funktionalität zur Löschung und Erstellung von neuen Nutzeraccounts sowie zusätzliche Funktionen wie Verwaltung von Freunden und Erfahrungspunkten bereit. Ein neuer Account wird dabei erstellt, falls beim Login über die OpenID Connect Schnittstelle mittels Authenticator der Nutzer noch nicht intern bekannt ist.

addUser(Username): Result $\langle Unit \rangle$

Nimmt einen Benutzernamen entgegen, der zuvor durch die OpenID Schnittstelle angelegt und authentifiziert wurde und speichert diesen in der Liste der aktiven Nutzer.

removeUser(Username): Result $\langle Unit \rangle$

Nimmt einen Benutzernamen entgegen und löscht diesen aus der Liste der Nutzer und löscht die mit ihm gespeicherten Informationen, falls er exisitert. Da jeder Nutzername eindeutig ist, wird maximal ein Nutzer aus der Liste gelöscht.

addFriend(Username1, Username2) : Result $\langle Unit \rangle$

Fügt einen Nutzer als Freund eines anderen Nutzers in der Datenbank

hinzu.

removeFriends(Username1, Username2) : Result $\langle Unit \rangle$
Löscht einen Nutzer als Freund des anderen Nutzers aus der Datenbank.

getExperiencePoints(Username) : Result $\langle Integer \rangle$ Gibt die Erfahrungspunkte eines gegebenen Nutzers zurück.

increaseExperiencePoints(Username, Points) : Result $\langle Unit \rangle$ Erhöht die Erfahrungspunkte eines gegebenen Nutzers um eine bestimmmte Anzahl in der Datenbank.

getUserList(): Result $\langle List\langle User \rangle \rangle$
Gibt die Liste der im System gespeicherten Nutzer zurück. Diese Liste ist aktuell.

Die Nutzer der App werden in der Klasse Account modelliert. Diese speichert den Benutzernamen, den Anzeigenamen sowie das XP-Level. Diese Daten werden in der Datenbank des Servers persistent gespeichert.

3.2.9 Datenbankzugriff

Der Zugriff auf die zugrundeliegende Datenbank erfolgt mittels der *Ktorm*-Bibliothek. Die Klasse DatabaseConnect wird dabei als Singleton implementiert, um einen konsistenten Zugriff auf die Datenbank sicherzustellen. Dabei wird die Verbindung zur Datenbank über das Config-Object festgelegt. Durch die methode connectDatabase kann der RequestController bei Start des HTTPS-Servers die Verbindung zur Datenbank herstellen

connectDatabase() : Database
Die Methode stellt eine Verbindung zur in dem Config-Objekt spezifizierten Datenbank her und gibt dieses Ktorm-Objekt zurück.

3.2.10 Diagramme

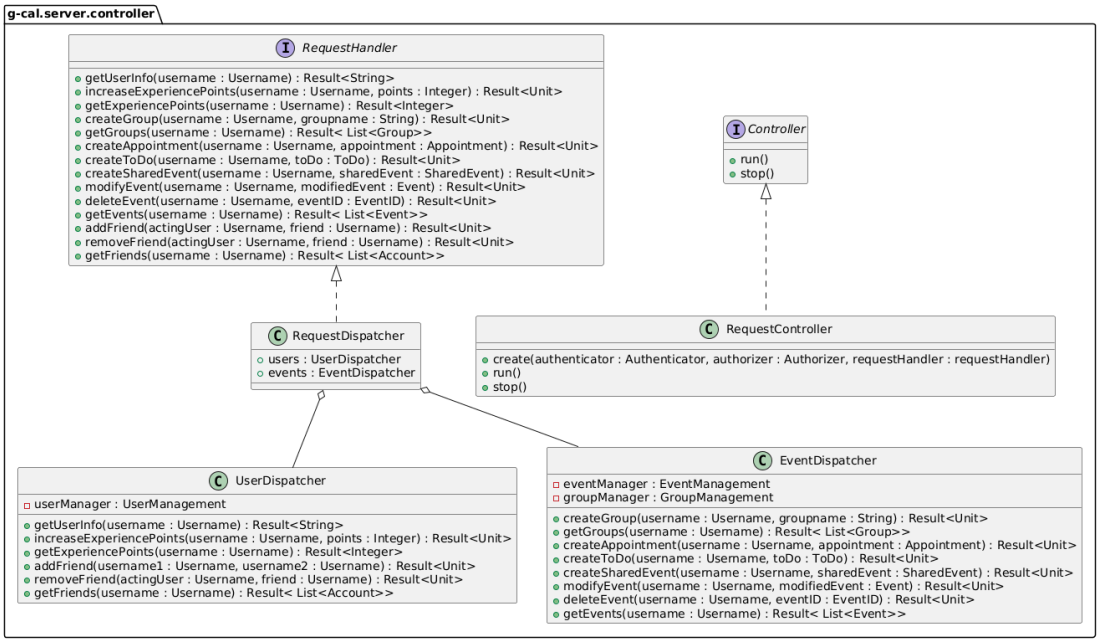


Abbildung 3.3: Beschreibung des Controller-Layers

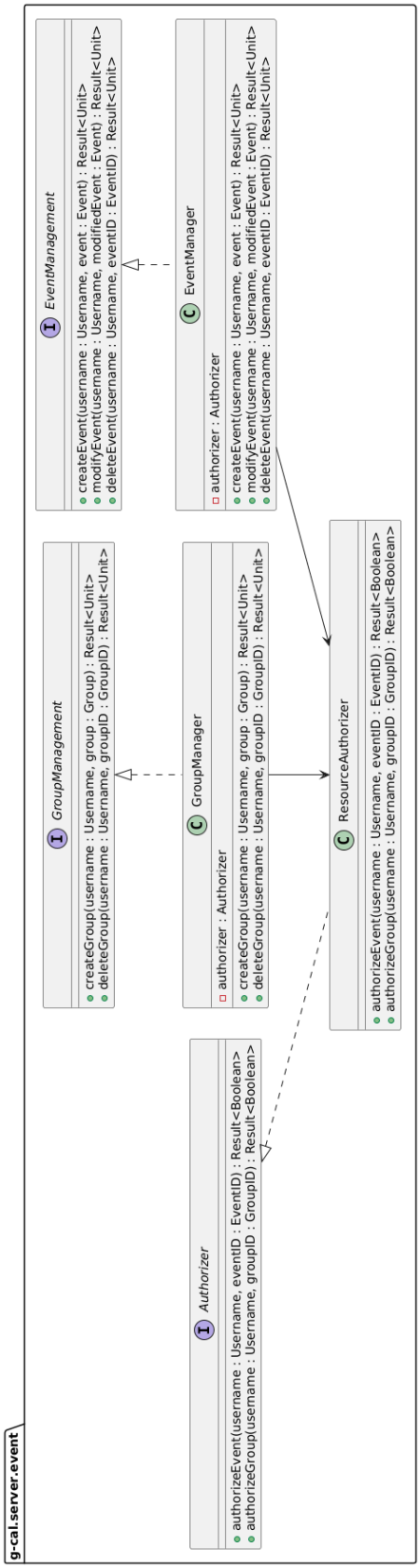


Abbildung 3.4: Klassen zur Event- und Gruppenverwaltung

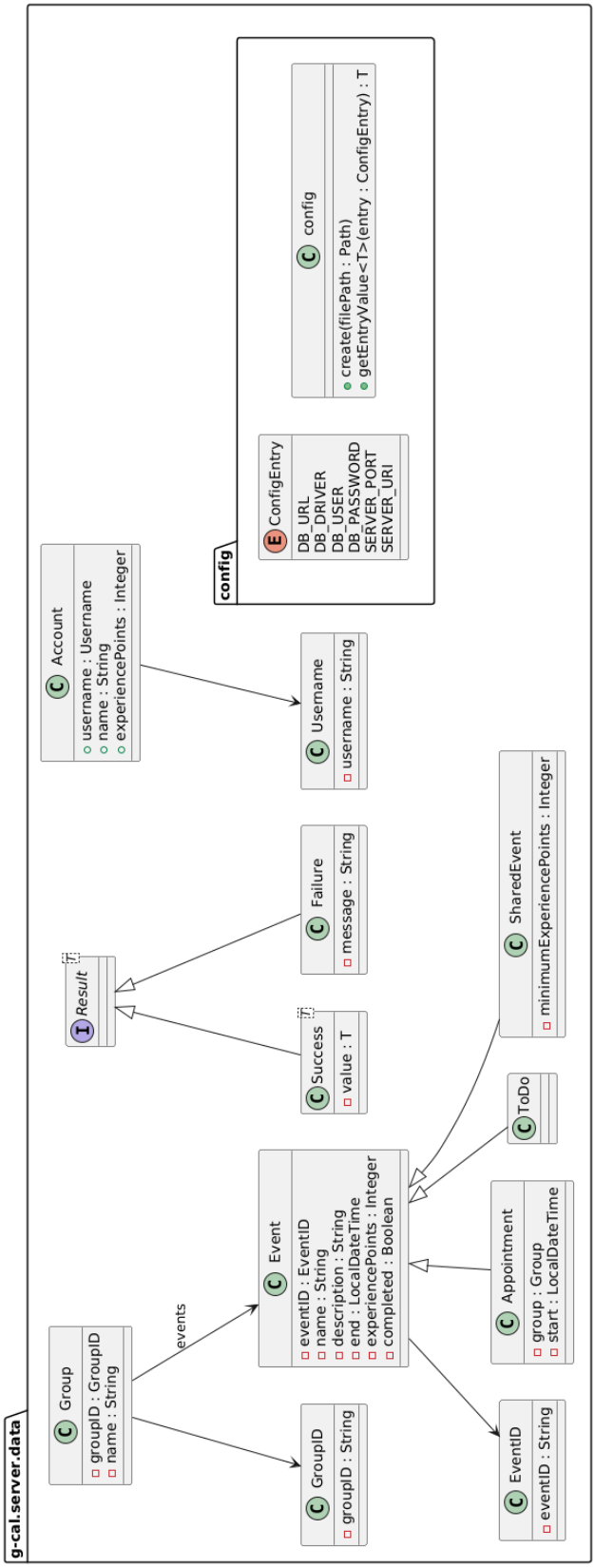


Abbildung 3.5: Die Datentypen des Servers

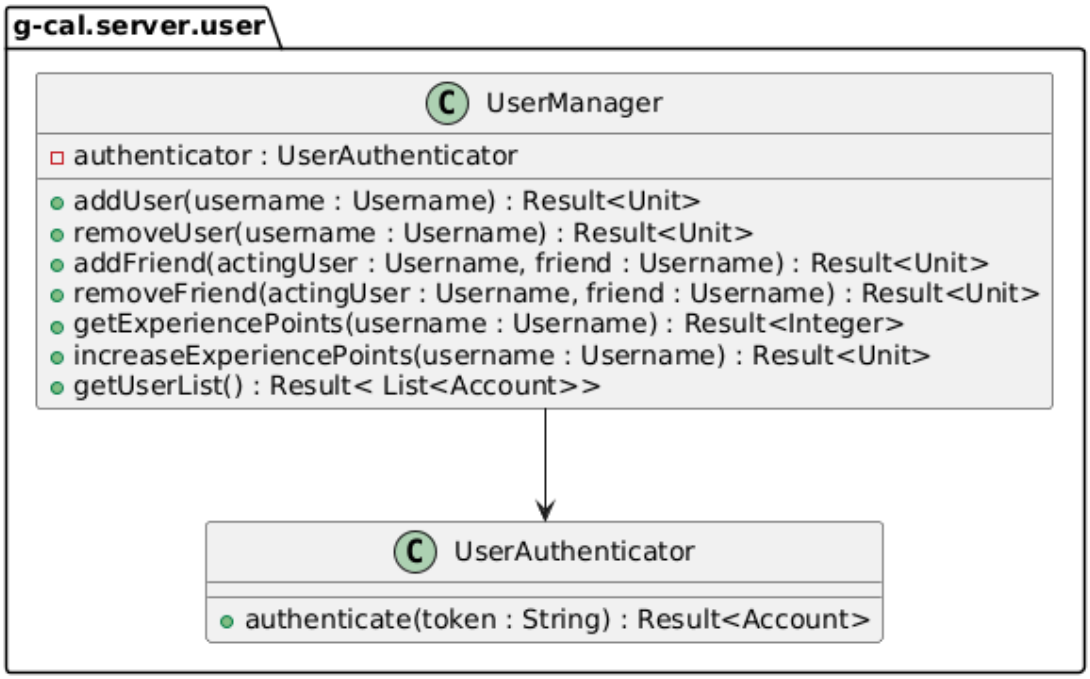


Abbildung 3.6: Klassen zur Nutzerverwaltung

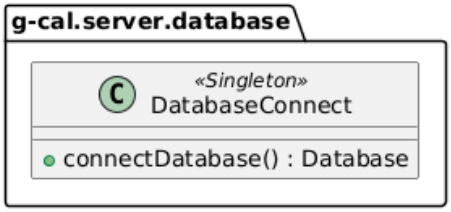


Abbildung 3.7: Schnittstelle zur Datenbankanbindung

3.3 Client Entwurf

3.3.1 Userinterface

3.3.1.1 Login Screen (Realisierung von F20) Der Login-Screen bildet den Einstiegspunkt der App. Er verantwortet die Authentifizierung via Google OpenID Connect (OIDC) sowie den Registrierungsprozess für Erstnutzer (Onboarding).

Funktionales Konzept (UI)
Layout-Architektur:
Der Screen nutzt ein zentriertes Layout innerhalb einer `Surface`. In diesem Screen wird bewusst auf das globale `Scaffold` und die `NavigationBar` verzichtet, um den Fokus auf die Anmeldung zu lenken.
View–ViewModel Schnittstelle (Unidirectional Data Flow):
Das `LoginViewModel` stellt den UI-Zustand als `StateFlow<LoginUiState>` bereit. Die `gruppenverwal`-Composable beobachtet den Zustand (z.B. via `collectAsStateWithLifecycle()`). Nutzerinteraktionen werden als Events/Methodenaufrufe an das ViewModel delegiert. Die Navigation wird als einmaliger Effekt (one-shot) ausgelöst.
Phase 1: Authentifizierung (Google OpenID Connect)
Initial befindet sich der Screen im Zustand `Unauthenticated`.

- **Branding:** App-Titel G-Cal und kurzer Hinweistext.
- **Interaktion:** Unterhalb des Titels befindet sich ein zentraler, rechteckiger Button mit der Aufschrift Login mit Google, der den OAuth2/OIDC Flow startet.
- **Technologie:** Der Flow wird über einen Chrome Custom Tab gestartet (sichere Anmeldung).
- **Loading-Feedback:** Während des Flows zeigt die UI einen Ladeindikator; der Button ist deaktiviert.

Nach erfolgreichem OIDC-Login prüft das System, ob für den Account bereits ein Benutzername existiert: Ist der Nutzer bekannt, navigiert die App direkt zum `MainScreen`. Andernfalls wird in den Onboarding-Modus gewechselt.

Phase 2: Onboarding (Benutzername-Erfassung)
Ist der Nutzer neu, wechselt die UI dynamisch (Conditional Rendering) in den Onboarding-Modus:

- Hinweistext Bitte wähle einen Benutzernamen.
- `OutlinedTextField` zur Eingabe (inkl. Inline-Validierung).
- `Weiter`-Button zum Abschließen der Registrierung; der Button ist nur aktiv, wenn die Eingabe clientseitig valide ist.

Benutzername (Validierung, V1):
Die Eingabe wird vor der Prüfung mit `trim()` bereinigt und anschließend in eine kanonische Form überführt (`lowercase(Locale.ROOT)`). Zulässig sind ausschließlich Kleinbuchstaben `a-z`, Ziffern `0-9` und der Unterstrich `_`. Die Länge muss zwischen 3 und 20 Zeichen liegen. Der Benutzername muss mit einem Buchstaben beginnen und mit einem Buchstaben oder einer Ziffer enden. Clientseitig wird zusätzlich folgende Regex geprüft:

```
^[a-z][a-z0-9_]{1,18}[a-z0-9]$
```

Die Eindeutigkeit wird serverseitig anhand der kanonischen Form geprüft. Reservierte Namen (z.B. `admin`, `root`) sind nicht zulässig.
Navigation:
Sobald der Nutzer authentifiziert ist und (falls nötig) einen Benutzernamen erfolgreich registriert hat, navigiert die App zum `MainScreen`. Dabei wird der Back-Stack bereinigt (*popUpTo inclusive*).
Fehler- und Sonderfälle (UI-Handling):

- **Abbruch durch Nutzer:** Rückkehr in den Zustand `Unauthenticated`.
- **Netzwerkfehler:** Anzeige einer Fehlermeldung (Error-Card oder Snackbar) mit `Retry`-Option.
- **Ungültiger Benutzername:** Inline-Fehler am Textfeld; `Weiter`-Button deaktiviert.

Technischer Klassentwurf

Dieser Abschnitt definiert die spezifischen Klassen und Datenstrukturen, die den oben beschriebenen Login-Flow realisieren.

1. Zustands-Steuerung (Enums)

Enum: ***`LoginUiStep`***

Definiert, welcher Inhalt im Hauptbereich angezeigt wird.

- *Werte:*
  - `START`: Der Nutzer ist nicht authentifiziert; zeigt den Login mit Google-Button.
  - `ONBOARDING`: Der Nutzer ist authentifiziert, muss aber noch einen Benutzernamen wählen.

2. Datenmodelle (View Data)

Klasse: ***`gruppenverwalState`***

Hält den vollständigen UI-Zustand des Screens.

- **Attribute:**
  - `currentStep: LoginUiStep` – Steuert die Sichtbarkeit von Login- oder Onboarding-Content.
  - `usernameInput: String` – Der aktuell eingegebene Text im Onboarding-Feld.
  - `usernameError: String?` – Fehlermeldung bei Validierungsproblemen.
  - `isSubmitEnabled: Boolean` – Gibt an, ob der Weiter-Button klickbar ist.
  - `isLoading: Boolean` – Steuert die Sichtbarkeit des Lade-Overlays.

3. UI-Komponenten (View Classes)

Klasse: ***`LoginScreenView`*** *(Root)*

Der zustandsbehaftete Container (Stateful), der das ViewModel verbindet.

- **Input:** `state: LoginScreenState`
- **Struktur (Scaffold):**
  - **Content:** Wechselt basierend auf `currentStep` zwischen `LoginContentView` und `OnboardingContentView`.
  - **Overlay:** Zeigt `LoadingOverlay`, wenn `isLoading == true`.

  - **Host:** Beinhaltet den `SnackbarHost` für Fehlermeldungen.

Klasse: `LoginContentView`
Die Startansicht für nicht-angemeldete Nutzer.

- **Elemente:**
  - App-Logo (Zentriert).
  - Rechteckiger Button Login mit Google.
- **Output:** Delegiert Klick an `onLoginClicked`.

Klasse: `OnboardingContentView`
Das Formular zur Eingabe des Benutzernamens.

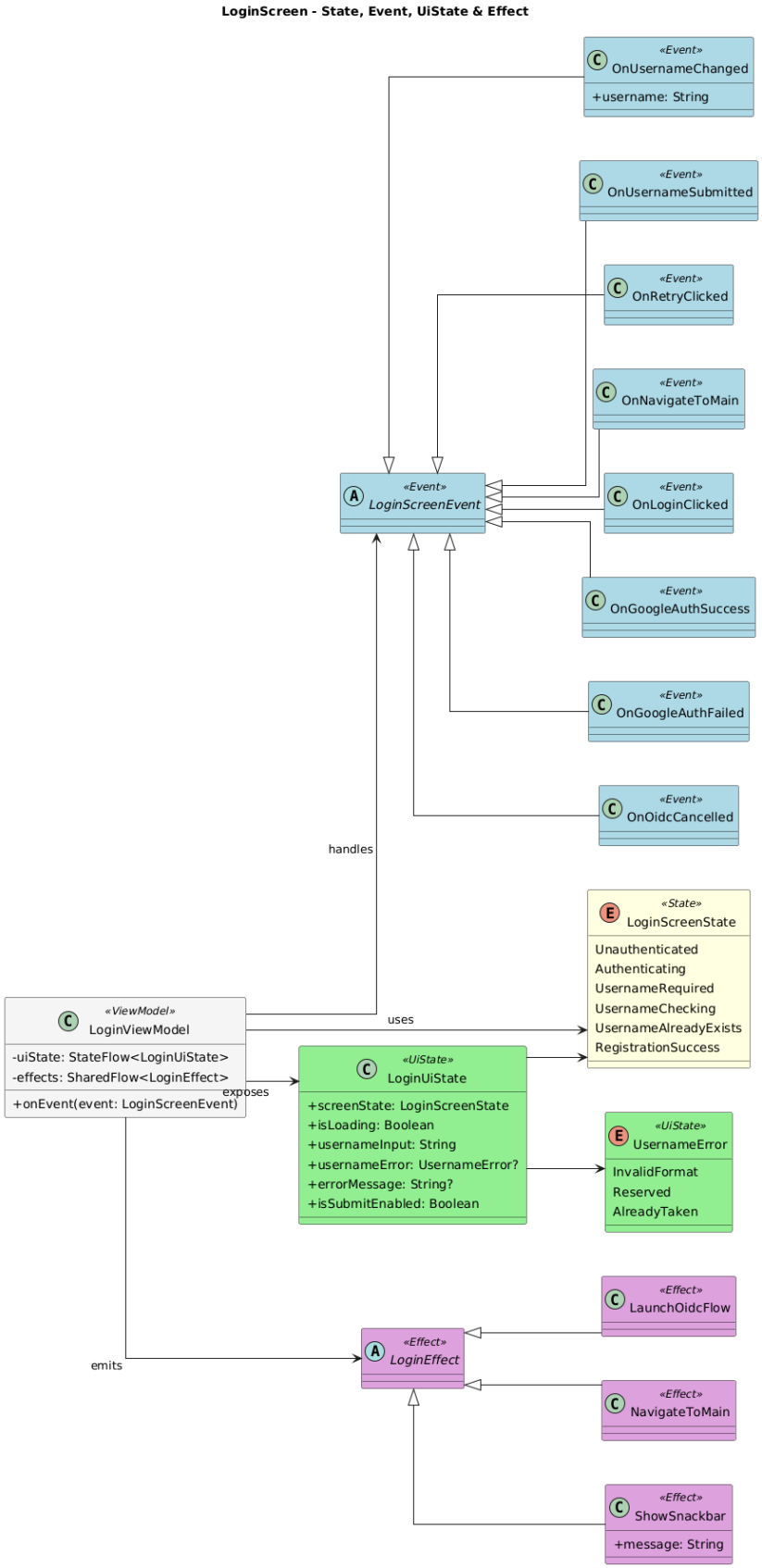
- **Input:** `username`, `errorMsg`, `isEnabled`.
- **Output:** `onUsernameChanged(text)`, `onSubmitClicked()`.

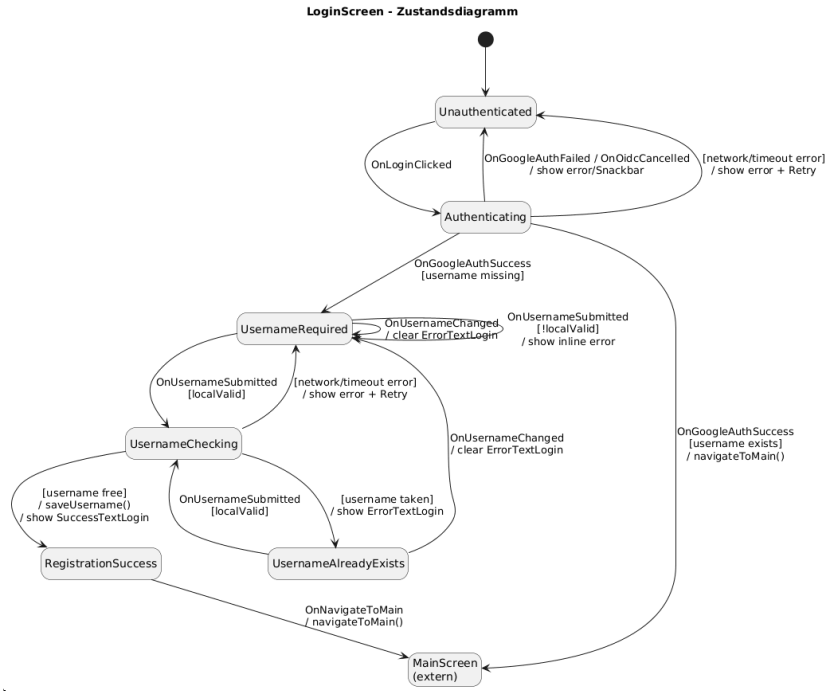
Klasse: `LoadingOverlay`
Ein halbtransparenter Hintergrund mit einem `CircularProgressIndicator`.

4. Schnittstellen (Interaction Listener)

Interface: `LoginInteractionListener`
Definiert die Events, die vom UI an die Logik (ViewModel) gesendet werden.

- **Authentifizierung:** `onLoginClicked()` (Startet OIDC-Flow).
- **Onboarding:** `onUsernameChanged(input)`, `onOnboardingSubmit()`.


LoginScreen - State, Event, UiState & Effect
«Event»
OnUsernameChanged
+username: String
«Event»
OnUsernameSubmitted
«Event»
OnRetryClicked
«Event»
OnNavigateToMain
«Event»
LoginScreenEvent
«Event»
OnLoginClicked
«Event»
OnGoogleAuthSuccess
«Event»
OnGoogleAuthFailed
«Event»
OnOidcCancelled
handles
«State»
LoginScreenState
Unauthenticated
Authenticating
UsernameRequired
UsernameChecking
UsernameAlreadyExists
RegistrationSuccess
«ViewModel»
LoginViewModel
-uiState: StateFlow<LoginUiState>
-effects: SharedFlow<LoginEffect>
+onEvent(event: LoginScreenEvent)
uses
exposes
«UiState»
LoginUiState
+screenState: LoginScreenState
+isLoading: Boolean
+usernameInput: String
+usernameError: UsernameError?
+errorMessage: String?
+isSubmitEnabled: Boolean
«UiState»
UsernameError
InvalidFormat
Reserved
AlreadyTaken
«Effect»
LaunchOidcFlow
emits
«Effect»
LoginEffect
«Effect»
NavigateToMain
«Effect»
ShowSnackbar
+message: String


LoginScreen - Zustandsdiagramm
Unauthenticated
OnLoginClicked
OnGoogleAuthFailed / OnOidcCancelled
/ show error/Snackbar
[network/timeout error]
/ show error + Retry
Authenticating
OnGoogleAuthSuccess
[username missing]
UsernameRequired
OnUsernameChanged
/ clear ErrorTextLogin
OnUsernameSubmitted
[!localValid]
/ show inline error
OnUsernameSubmitted
[localValid]
[network/timeout error]
/ show error + Retry
UsernameChecking
OnUsernameChanged
/ clear ErrorTextLogin
OnGoogleAuthSuccess
[username exists]
/ navigateToMain()
[username free]
/ saveUsername()
/ show SuccessTextLogin
OnUsernameSubmitted
[localValid]
[username taken]
/ show ErrorTextLogin
RegistrationSuccess
UsernameAlreadyExists
OnNavigateToMain
/ navigateToMain()
MainScreen
(extern)

3.3.1.2 Main Screen: Kalender-Hub & Event-Management Der Main-Screen bildet das funktionale Zentrum der Anwendung. Er implementiert die zentralen Anforderungen an die Zeitverwaltung (F70, F120), die Organisation in Gruppen (F50, F60) sowie die Manipulation von Datensätzen (F80, F90, F100). Das UI-Konzept folgt den Prinzipien des *Material Design 3* und priorisiert eine inhaltszentrierte Darstellung.

Funktionales Konzept (UI)

1. Layout-Struktur & Navigation

Der Screen basiert auf einem `Scaffold`-Layout, das in drei Hauptbereiche unterteilt ist:

- **TopAppBar (Steuerleiste):** Der obere Bereich besteht aus insgesamt sechs klickbaren Elementen, die in zwei Gruppen unterteilt sind:
  - **Ansichts-Wahl (4 Elemente):** Links befinden sich die Umschalter für die Modi *Tag, Woche, Monat, Jahr*. Die aktuell aktive Ansicht wird farblich hervorgehoben, während die inaktiven Ansichten visuell ausgegraut sind, um dem Nutzer eine klare Orientierung zu bieten.
  - **Aktionen (2 Elemente):** Rechtsbündig befinden sich das Icon für die Gruppen-Filterung (F60) und die Suche (Jump-to-Date, F120).
- **Synchronisation:** Die Navigation ist bidirektional. Ein Wechsel der Ansicht über die TopAppBar aktualisiert den Kalenderbereich. Umgekehrt aktualisiert eine Wischgeste (Swipe) im Kalenderbereich automatisch die Datumsanzeige und den Status in der TopAppBar.
- **Floating Action Button (FAB):** Ein fixierter Button (+) in der unteren rechten Ecke öffnet das Erstellungs-Menü (BottomSheet).

Globale Navigation (Bottom Bar): Am unteren Bildschirmrand befindet sich eine permanente Navigationsleiste (`BottomNavigationBar`). Über diese kann der Nutzer jederzeit zwischen den Hauptbereichen *Rangliste, Berichte* und *Profil* wechseln. Der Kalender fungiert dabei als Startansicht (Home), zu der man über eine Rück-Navigation von den anderen Tabs zurückkehrt.

2. Kalender-Ansichten (Realisierung von F70, F130)

Der Content-Bereich passt sich dynamisch dem gewählten Modus an:

- **Tag (Day View):** Zeigt eine chronologische Liste aller Einträge (Events, ToDos, SharedEvents) des gewählten Tages.

  - **Sticky Header:** Das aktuelle Datum (z.B. 8. Januar 2025) bleibt am oberen Rand fixiert, auch wenn der Nutzer durch die Liste scrollt.
  - **Karten-Design:** Jeder Eintrag wird als Karte dargestellt.

- **Woche (Week View):** Eine vereinfachte Übersicht der Wochentage ohne starren Zeitstrahl (Timeline), um Komplexität zu reduzieren. Ein Klick auf einen Tag navigiert direkt in die Tagesansicht.

- **Monat (Month View):** Ein klassisches Raster-Layout. Der aktuelle Tag ist farblich markiert.

  - **Navigation:** Durch horizontale Wischgesten kann zwischen den Monaten (z.B. Januar ↔ Februar) gewechselt werden. Über dem Raster wird der Name des Monats angezeigt.

- **Jahr (Year View):** Eine Übersicht der 12 Monate. Auch hier ermöglicht eine Wischgeste den Wechsel zwischen den Jahren (z.B. 2025 ↔ 2026). Ein Klick auf einen Monat öffnet die Monatsansicht.

3. Event-Management (Realisierung von F70, F80, F90)

Die Datenmanipulation erfolgt über ein zentrales *Modal Bottom Sheet*. Dieses Formular dient sowohl der Erstellung neuer als auch der Bearbeitung bestehender Einträge.

- **Struktur des Formulars:** Das Layout ist in drei Tabs unterteilt, deren Eingabefelder sich je nach Typ ändern:

  - **ToDo:** Name, Datum, Uhrzeit, Gruppe,.
  - **Termin:** Zusätzlich *Standort*.
  - **SharedEvent:** Zusätzlich *Standort*, *Bemerkung*, *Rang-Level* (manuell) und *Teilnehmer-Liste* (Teilen mit...).

- **Wiederholungs-Optionen (Replay):** In jedem Tab stehen vier Buttons zur Wahl: *Täglich, Wöchentlich, Monatlich, Jährlich*. Standardmäßig ist keine Wiederholung aktiviert (Einmaliges Event).

- **Modus: Erstellen (Create):** Bei der Neuanlage (via FAB) öffnet sich das Formular mit leeren Feldern. Im Footer befindet sich der Button Speichern.

- **Modus: Bearbeiten (Edit) & Löschen:** Ein Klick auf eine bestehende Karte öffnet das Formular mit vorausgefüllten Daten.
  - **Sicheres Löschen:** Der Footer enthält zusätzlich einen Löschen-Button, der eine Bestätigung fordert (*AlertDialog*).
  - **Serien-Löschung:** Bei wiederkehrenden Terminen wird gefragt, ob nur dieser Termin oder alle zukünftigen Termine der Serie gelöscht werden sollen.
- **Navigation:** Jedes Dialog-Fenster kann über ein X oben links geschlossen werden, ohne Änderungen zu speichern.

4. Gamification Abhaken (F100)
Um den spielerischen Aspekt zu fördern:

- **Checkbox:** Jede Karte (in allen Listenansichten) verfügt in der oberen linken Ecke über eine Checkbox (Abhaken).
- **XP-Gewinn:** Das Markieren als erledigt ändert den visuellen Status der Karte (ausgegraut/durchgestrichen) und schreibt dem Nutzer die definierten XP gut.

5. Gruppenverwaltung Filterung (Realisierung von F50 ,F60)
Die Filterung erfolgt über einen mehrstufigen Dialog-Prozess:

- **Filter-Dialog:** Zeigt eine Liste aller Gruppen mit farbigen Boxen am linken Rand. Diese Boxen können mit einem X markiert werden, um die Gruppe auszublenden.
- **Gruppe Erstellen:** Ein Button Gruppe Erstellen am unteren Rand öffnet einen zweiten Dialog (Overlay). Hier können *Name* und *Farbe* definiert werden.
- **Rück-Navigation:** Das Schließen des Erstellungs-Dialogs (über X) führt zurück zum Filter-Dialog, nicht zum Hauptbildschirm.

6. Fehlerbehandlung Validierung
Das System fängt potenzielle Fehleingaben und technische Probleme ab:

- **Input-Validierung:** Das Speichern ist deaktiviert, wenn Pflichtfelder (z.B. Name) leer sind oder unlogische Zeiten (Ende vor Start) eingegeben wurden. Negative XP-Werte werden blockiert.

- **Netzwerk Daten:** Bei Verbindungsabbrüchen während der Erstellung von SharedEvents wird der Nutzer benachrichtigt.
- **Sicherheitsabfragen:** Kritische Aktionen wie Löschen oder das Verwerfen von ungespeicherten Änderungen werden durch Bestätigungsdialoge abgesichert.

Technischer Klassentwurf

Dieser Abschnitt definiert die Navigationsleiste, die interaktiven Overlay-Fenster und die Kalenderlogik gemäß dem spezifizierten User-Flow.

1. Top-Bar und Dialog-System

Klasse: `CalendarTopBar`

Die Steuerleiste im oberen Bereich des Kalenders.

- **Layout:** Besteht aus 6 interaktiven Elementen.
- **Attribute (Input):**
  - `activeMode: ViewMode` – Steuert die Opazität der 4 Ansichts-Buttons (Tag, Woche, Monat, Jahr).
- **Events (Output):**
  - `onViewModeSelected(mode: ViewMode)` – Wechselt die Kalenderansicht.
  - `onFilterClicked()` – Öffnet den `FilterDialog`.
  - `onJumpToDateClicked()` – Öffnet den `JumpToDateDialog`.

Dialog-System (Overlays)

Alle Dialoge besitzen standardmäßig einen X-Button oben links zum Schließen (`onCloseClicked`).

Klasse: `FilterDialogView` (Overlay 1)

Zeigt die Liste der Gruppen zum Ein-/Ausblenden.

- **Input:**
  - `groups: List<UiFilterGroup>` – Die Liste der existierenden Gruppen. Ist die Liste leer, wird nichts angezeigt.

- **Elemente:**
  - Liste mit Checkboxen (oder X-Markierungen).
  - Button Gruppe Erstellen am unteren Rand.
  - Button X (Schließen) oben links.
- **Output:**
  - `onToggleGroup(id: String)` – Gruppe filtern.
  - `onOpenCreateGroupClicked()` – Öffnet den `CreateGroupDialog` (Overlay 2).
  - `onCloseClicked()` – Schließt den Dialog.

Klasse: `CreateGroupDialogView` (Overlay 2)
Ermöglicht das Erstellen einer neuen Gruppe.

- **Input:**
  - `availableColors: List<String>` – Eine vorgegebene Palette an Farbcodes (Hex) zur Auswahl.
- **Eingabefelder:**
  - `nameInput: String` – Textfeld für den Gruppennamen.
  - `selectedColor: String` – Die aus der Palette gewählte Farbe.
- **Aktionen:**
  - Button Speichern (unten).
  - Button X (Schließen/Abbrechen) oben links.
- **Output:**
  - `onSaveGroup(name: String, color: String)` – Speichert die Gruppe und kehrt zu Overlay 1 zurück.
  - `onCloseClicked()` – Kehrt zu Overlay 1 zurück ohne zu speichern.

Klasse: `JumpToDateDialogView` (Overlay 3)
Ermöglicht den direkten Sprung zu einem Datum.

- **Eingabefelder:**

  - `dateInput: DatePicker` – Auswahlfeld für das Zieldatum.

- **Aktionen:**
  - Rechteckiger Button Gehe zu Datum (Go to Date).
  - Button X (Schließen) oben links.

- **Output:**
  - `onConfirmDate(date: LocalDate)` – Aktualisiert den Kalender auf das gewählte Datum und schließt den Dialog.
  - `onCloseClicked()` – Schließt den Dialog.

2. Spezifische Kalender-Ansichten

Der mittlere Bereich (`ContentArea`) passt sich dynamisch dem gewählten `ViewMode` an. Die Darstellung basiert auf echter Kalenderlogik (Montag bis Sonntag).

Klasse: `CalendarHeader`

Eine Text-Komponente, die sich direkt unter der TopBar befindet und den angezeigten Zeitraum beschreibt.

- **Input:** `currentDate: LocalDate`, `mode: ViewMode`
- **Verhalten (Formatierung):**
  - *Bei Tag:* Zeigt 9. Januar 2026.
  - *Bei Woche/Monat:* Zeigt Januar 2026.
  - *Bei Jahr:* Zeigt 2026.

Klasse: `DayViewList` (Tagesansicht)

Zeigt den detaillierten Tagesablauf.

- **Datenquelle:** Eine Liste von `UiEventEntry`-Objekten.
- **Sortierlogik:** Die Einträge werden strikt chronologisch nach ihrer Startzeit sortiert (00:00 → 23:59).
- **Visuelle Darstellung:**
  - Jeder Eintrag wird als Karte gerendert.
  - **Farb-Indikator:** Die Farbe der zugehörigen Gruppe wird als vertikaler Streifen am linken Rand der Karte angezeigt (Simple Border Strategy).

- **Interaktion:** Klick auf Karte → Öffnet Editor.

Klasse: `WeekViewGrid` (Wochenansicht)
Eine komprimierte Übersicht der Woche (Montag bis Sonntag).

- **Layout:** 7 Spalten (für die Wochentage).
- **Inhalt:**
  - Zeigt pro Tag eine Liste von Mini-Karten.
  - **Reduktion:** Um Platz zu sparen, wird nur der Titel des Events angezeigt (keine Uhrzeit, kein Ort).
- **Header:** Über den Spalten stehen die Wochentage (Mo, Di, Mi...).
- **Interaktion:** Klick auf einen Spaltenkopf (Tag) → Wechselt in die `DayViewList` dieses Tages.

Klasse: `MonthViewGrid` (Monatsansicht)
Das klassische Kalenderraster.

- **Layout:** Raster mit max. 6 Zeilen und 7 Spalten.
- **Aktueller Tag:** Das Feld, das dem heutigen Datum entspricht (Systemdatum), wird mit einer roten Umrandung oder roter Zahl hervorgehoben.
- **Inhalt pro Zelle:** Zeigt kleine Punkte (Event-Marker) in den Farben der Gruppen, um die Belegung zu signalisieren.
- **Interaktion:** Klick auf eine Zelle (Tag) → Wechselt in die `DayViewList` des gewählten Tages.

Klasse: `YearViewGrid` (Jahresansicht)
Die höchste Abstraktionsebene.

- **Layout:** 3x4 oder 4x3 Raster (Januar bis Dezember).
- **Inhalt:** Zeigt nur die Monatsnamen (z.B. Februar).
- **Interaktion:** Klick auf einen Monat → Wechselt in die `MonthViewGrid` des gewählten Monats.

Erweiterung des InteractionListener
Um die Navigation zwischen den Ansichten zu ermöglichen:

- `onDaySelected(date: LocalDate)`: Wird aufgerufen, wenn im Monats- oder Wochenraster auf einen Tag geklickt wird. *Logik: Setzt Datum und wechselt Mode auf DAY.*
- `onMonthSelected(yearMonth: YearMonth)`: Wird aufgerufen, wenn im Jahresraster auf einen Monat geklickt wird. *Logik: Wechselt Mode auf MONTH.*

3. Erstellung von Einträgen (Creation Flow)

Der Prozess zum Erstellen neuer Einträge wird über einen permanent sichtbaren Floating Action Button eingeleitet, der eine dedizierte Unterseite (*Subscreen*) öffnet.

Erweiterung: `MainScreenView`
Die bestehende Ansicht wird um einen Aktions-Button erweitert.

- **Element:** `FloatingActionButton (FAB)`
  - **Position:** Fixiert in der unteren rechten Ecke, oberhalb der BottomBar.
  - **Symbol:** Ein Plus-Icon (+).
  - **Interaktion:** Ein Klick löst `onOpenCreateScreenClicked()` aus.

Klasse: `CreateEntryView` (Subscreen)
Eine eigenständige Vollbild-Ansicht zum Erstellen von ToDos, Terminen und SharedEvents.

- **Input:**
  - `draftState: EntryDraftState` – Der aktuelle Zustand der Eingabefelder.
  - `availableGroups: List<Group>` – Für das Dropdown-Menü der Gruppenauswahl.
  - `friendList: List<User>` – Für die Teilnehmerauswahl (nur SharedEvent).
- **Struktur (Layout):**
  - **Top-Bar:**
    - *Links:* Zurück-Pfeil (Bricht ab und kehrt zum MainScreen zurück).
    - *Titel:* Neuer Eintrag.

  - **Typ-Auswahl (3 Buttons):** Eine Leiste mit drei Schaltflächen: `ToDo`, `Termin`, `SharedEvent`. Der gewählte Typ ändert die sichtbaren Eingabefelder dynamisch.
  - **Dynamisches Formular (Body):**
    * *Basis-Felder (Alle Typen):* Name (Text), Datum (Picker), Uhrzeit (Picker), Gruppe (Dropdown).
    * *Zusatz bei Termin:* Standort (Text/Map).
    * *Zusatz bei SharedEvent:* Standort, Bemerkung (Textarea), Rang-Level (Input), Teilen mit... (Multi-Select Liste).
  - **Footer:**
    * Button Speichern (Rechteckig, volle Breite). Speichert den Eintrag und navigiert zurück.

- **Output (Events):**
  - `onTypeChanged(type: EntryType)` – Wechselt zwischen den 3 Vorlagen.
  - `onInputValueChanged(field: FieldType, value: Any)` – Aktualisiert den Draft-State.
  - `onSaveClicked()` – Validiert und speichert.
  - `onBackClicked()` – Bricht ab.

Klasse: ***`EntryDraftState`***
Hält die temporären Daten während der Eingabe.

- **Attribute:**
  - `selectedType: EntryType` – Default: `ToDo`.
  - `name: String`
  - `date: LocalDate`, `time: LocalTime`
  - `selectedGroup: Group?`
  - `xpLevel: Int`
  - `location: String?` (Nullable: nur für Termin/Shared).
  - `note: String?` (Nullable: nur für Shared).
  - `rankLevel: Int?` (Nullable: nur für Shared).

  - `sharedWith: List<User>?` (Nullable: nur für Shared).

Erweiterung des InteractionListener
Methoden für den Erstellungs-Flow:

- `onOpenCreateScreenClicked()`: Wechselt vom MainScreen in den `CreateEntryView`.
- `onSaveNewEntry(draft: EntryDraftState)`: Speichert die Daten in der Datenbank und schließt den Screen.
- `onCancelCreation()`: Schließt den Screen ohne zu speichern (Zurück-Pfeil).

4. Bearbeiten und Löschen von Einträgen

Der Bearbeitungsmodus ermöglicht das Modifizieren bestehender Daten sowie das Entfernen von Einträgen. Hierbei wird derselbe Dialog wie bei der Erstellung verwendet, jedoch in einem anderen Zustand (*State*).

Erweiterung der Enumerationen
Um zwischen Erstellen und Bearbeiten zu unterscheiden:

`enum EditorMode`

Steuert das Verhalten des Editor-Dialogs.

- *Werte:*
  - `CREATE`: Leeres Formular, nur Speichern-Button.
  - `EDIT`: Vorausgefülltes Formular, Speichern und Löschen Buttons.

Klasse: `EntryEditorDialog` (Unified Editor)
Dieser Dialog dient sowohl der Erstellung als auch der Bearbeitung.

- **Input (State):**
  - `mode: EditorMode` – Bestimmt das Layout der Buttons.
  - `entryData: EntryDraftState` – Die Daten des Eintrags.
    - *Bei CREATE:* Standardwerte (leer).
    - *Bei EDIT:* Die Daten der angeklickten Karte (Titel, Zeit, Gruppe etc.) sind bereits eingetragen.
- **Layout-Anpassung (Footer):** Im `EDIT`-Modus werden am unteren Rand zwei rechteckige Buttons untereinander (vertikal gestapelt) angezeigt:

1. **Button Speichern:** Aktualisiert die Änderungen.
2. **Button Löschen:** Öffnet den `DeleteConfirmationDialog`.

- **Output:**
  - `onUpdateEntry(updatedData: EntryDraftState)` – Speichert Änderungen.
  - `onRequestDelete(isRecurring: Boolean)` – Signalisiert Löschabsicht und prüft, ob es ein Serientermin ist.

Klasse: `DeleteConfirmationDialog` (Overlay)
Ein Sicherheits-Dialog, der erscheint, wenn im Editor auf Löschen geklickt wird. Er passt seinen Inhalt dynamisch an die Art des Termins an.

- **Input:**
  - `isRecurringSeries: Boolean` – Handelt es sich um ein wiederkehrendes Event?
- **Logik Anzeige:**
  - **Fall A (Keine Wiederholung):**
    - *Text:* Möchten Sie diesen Eintrag wirklich löschen?
    - *Buttons:* Abbrechen vs. Löschen.
  - **Fall B (Wiederholung / Serie):**
    - *Text:* Dies ist ein wiederkehrender Eintrag.
    - *Buttons:*
      1. Nur diesen Eintrag löschen (Löscht nur den heutigen Tag).
      2. Alle zukünftigen löschen (Löscht die gesamte Serie ab heute).
- **Output:**
  - `onConfirmDelete(scope: DeleteScope)` – Führt die Löschung aus.
  - `onCancel()` – Schließt den Dialog ohne Aktion.

Hilfs-Enum: `DeleteScope`

- *Werte:* `SINGLE_INSTANCE` (Nur einer), `SERIES_FUTURE` (Alle zukünftigen).

Erweiterung des InteractionListener
Methoden für die Datenmanipulation:

- `onEntryClicked(id: String)`: Lädt die Daten des Eintrags und öffnet den `EntryEditorDialog` im Modus `EDIT`.
- `onUpdateEntryClicked(data: EntryDraftState)`: Speichert die Änderungen und schließt den Editor.
- `onDeleteConfirmed(id: String, scope: DeleteScope)`: Führt die endgültige Löschung in der Datenbank aus und aktualisiert den Main-Screen.

3.3.1.3 RanglisteScreen - (Realisierung von F160) Dieser Screen visualisiert die globale Rangliste der Nutzer und fördert den Wettbewerb durch Gamification-Elemente. Der Entwurf setzt die Anforderungen an eine übersichtliche und performante Listenansicht um.

Funktionales Konzept (UI)
Einbindung in Navigation & BottomNavigationBar:
Der Screen ist über die BottomNavigationBar erreichbar. Beim Wechsel auf den Tab Rangliste wird dieser im `selected`-Zustand hervorgehoben, während die anderen Tabs (z.B. Berichte und Profil) in einer dezenteren Farbe dargestellt werden. Dadurch erkennt der Nutzer jederzeit eindeutig den aktuell aktiven Screen.
Layout-Struktur:
Der Screen verwendet ein `Scaffold`, um die grundlegende Struktur inklusive BottomNavigationBar bereitzustellen. Oben befindet sich eine `TopAppBar` mit einem Zurück-Button (Pfeil oben links). Im Content-Bereich wird zusätzlich eine leicht links ausgerichtete Überschrift Rangliste dargestellt (visuell wie in der UI-Skizze), um den Screen eindeutig zu kennzeichnen.
Wichtige UI-Komponenten:

- `LazyColumn` für Performance: Da die Rangliste dynamisch wachsen kann (potenziell hunderte Einträge), wird statt einer einfachen `Column` zwingend eine `LazyColumn` verwendet. Diese rendert nur die aktuell sichtbaren Elemente (Lazy Rendering) und ermöglicht flüssiges Scrollen.
- `LeaderboardItem`: Jeder Benutzer wird in einer `Card` dargestellt, um die Daten visuell zu gruppieren. Die Liste ist primär absteigend nach dem

Rang-Level sortiert. Innerhalb der Card ordnet eine `Row` die Informationen horizontal an:

- Platzierung (Rang): Ein fettgedrucktes Text-Element am linken Rand zeigt den Rang (z.B. #1).
- Name: Der Benutzername wird gut lesbar angezeigt.
- Level-Visualisierung: Das Level wird grafisch durch Sterne visualisiert. Hierfür wird eine `Row` verwendet, die basierend auf dem Level-Wert eine entsprechende Anzahl an Stern-Icons dynamisch generiert.
- XP-Anzeige (dynamischer Balken): Rechts wird der aktuelle XP-Wert in einem horizontalen Balken dargestellt, wobei der Wert direkt im Balken angezeigt wird (z.B. XP: 2450). Zusätzlich visualisiert die Füllung des Balkens den **Fortschritt zum nächsten Level**. Der Balken zeigt prozentual, wie viel XP noch zum Aufstieg fehlen (z.B. 50% gefüllt, wenn die Hälfte der benötigten XP erreicht ist).

Technischer Klassentwurf

Der `LeaderboardScreen` ist eine eigenständige Komponente, die über die BottomNavigationBar des MainScreens geladen wird. Er realisiert die Anforderung F160 (Globale Rangliste) und verwaltet seinen eigenen UI-Zustand unabhängig vom Kalender.

1. Zustands-Definitionen

Enum: ***`RanglisteScreenState`***

Definiert den aktuellen Ladezustand der Ansicht.

- *Werte:*
  - `Loading`: Der Ladevorgang läuft; UI zeigt einen Fortschrittsindikator.
  - `Loaded`: Daten erfolgreich abgerufen; UI zeigt die Liste der Nutzer.
  - `Error`: Ein Fehler ist aufgetreten; UI zeigt Fehlermeldung und Retry-Button.

Sealed Class: ***`RanglisteScreenEvent`***

Beschreibt alle Benutzeraktionen und Trigger, die von diesem Screen ausgehen.

- *Varianten:*
  - `OnScreenEntered`: Signalisiert, dass der Screen sichtbar wurde (startet das Laden).
  - `OnBackClicked`: Nutzer betätigt den Zurück-Pfeil (Navigation zum MainScreen).
  - `OnRetryClicked`: Nutzer fordert nach einem Fehler einen erneuten Ladeversuch an.

2. Datenmodelle (View Data)

Klasse: `LeaderboardEntryUi`

Repräsentiert die visuellen Daten einer einzelnen Nutzer-Karte in der Liste.

- **Attribute:**
  - `rank: Int` – Die Platzierung (z.B. 1, 2...).
  - `username: String` – Der Name des Nutzers.
  - `level: Int` – Das Rang-Level (bestimmt die Sortierung und Sterne).
  - `starCount: Int` – Anzahl der Sterne für die Level-Visualisierung.
  - `xpDisplayString: String` – Formatierter Text (z.B. XP: 2450).
  - `nextLevelProgress: Float` – Normalisierter Wert für den Balken ($0.0 - 1.0$), der den Fortschritt zum nächsten Level angibt.

Klasse: `RanglisteUiState`

Hält die dynamischen Daten für den `Loaded`-Zustand.

- **Attribute:**
  - `entries: List<LeaderboardEntryUi>` – Die sortierte Rangliste.
  - `userRankIndex: Int` – Index des aktuellen Nutzers in der Liste (für Hervorhebung).

3. UI-Komponenten (View Classes)

Klasse: `LeaderboardScreen` (Main View)

Die Hauptansicht für den Ranglisten-Tab.

- **Input:** `state: RanglisteScreenState`, `data: RanglisteUiState?`
- **Struktur (Scaffold):**

  - **TopBar:** Eine `TopAppBar` mit Titel Rangliste und Zurück-Pfeil (links).
  - **Content:**
    * *State == Loading:* Zeigt `CircularProgressIndicator`.
    * *State == Error:* Zeigt `ErrorView` (Text + Button).
    * *State == Loaded:* Zeigt `LeaderboardList`.

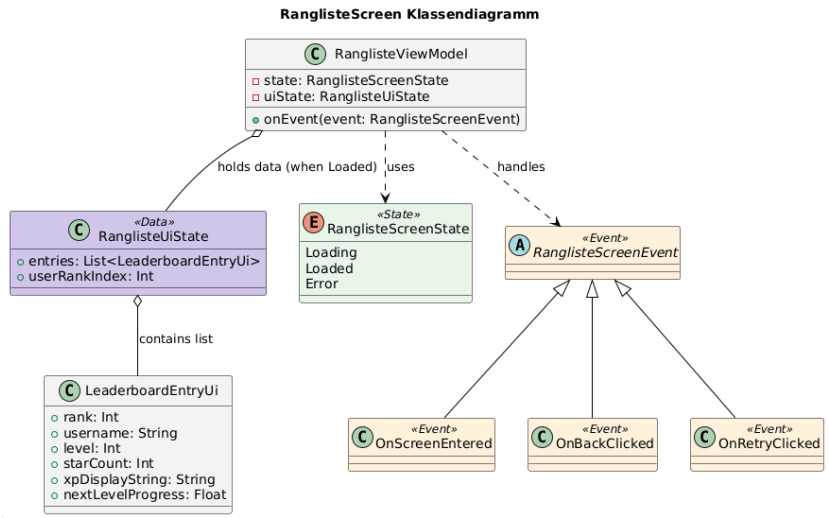
- **Output:** Sendet `RanglisteScreenEvent` an das ViewModel.

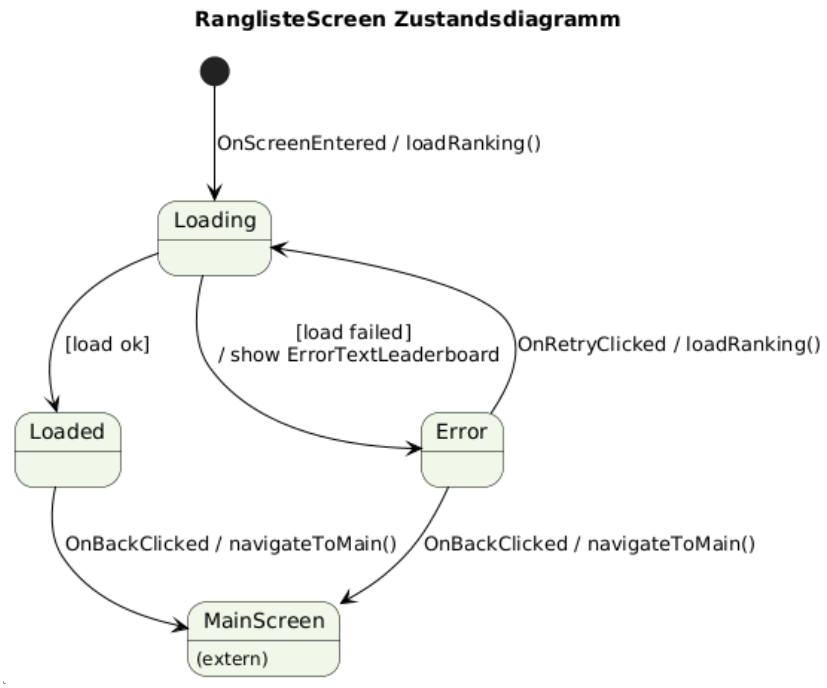
Klasse: `LeaderboardList`
Eine performante Listenkomponente.

- **Technologie:** Verwendet `LazyColumn` für Lazy Rendering (rendert nur sichtbare Elemente).
- **Inhalt:** Eine vertikale Liste von `LeaderboardItem`-Komponenten.

Klasse: `LeaderboardItem`
Die visuelle Karte für einen Eintrag.

- **Input:** `entry: LeaderboardEntryUi`
- **Layout:** Eine `Card`, die eine `Row` enthält:
  - **Links:** Platzierung (Fettgedruckter Text).
  - **Mitte:** Name und Level-Sterne (`Row` aus Icons).
  - **Rechts:** XP-Balken (`LinearProgressIndicator`) mit Text-Overlay.


RanglisteScreen Klassendiagramm
RanglisteViewModel
state: RanglisteScreenState
uiState: RanglisteUiState
onEvent(event: RanglisteScreenEvent)
holds data (when Loaded)
uses
handles
«Data»
RanglisteUiState
entries: List<LeaderboardEntryUi>
userRankIndex: Int
«State»
RanglisteScreenState
Loading
Loaded
Error
«Event»
RanglisteScreenEvent
contains list
LeaderboardEntryUi
rank: Int
username: String
level: Int
starCount: Int
xpDisplayString: String
nextLevelProgress: Float
«Event»
OnScreenEntered
«Event»
OnBackClicked
«Event»
OnRetryClicked


RanglisteScreen Zustandsdiagramm
OnScreenEntered / loadRanking()
Loading
[load ok]
[load failed]
/ show ErrorTextLeaderboard
OnRetryClicked / loadRanking()
Loaded
Error
OnBackClicked / navigateToMain()
OnBackClicked / navigateToMain()
MainScreen
(extern)

3.3.1.4 BerichteScreen (Realisierung von F140) Dieser Screen ermöglicht es dem Nutzer, Statistiken über erledigte und offene Aufgaben sowie die gesammelten XP für einen definierten Zeitraum einzusehen (F140). Der Entwurf teilt die Interaktion in zwei logische Phasen: (1) Auswahl des Zeitraums und (2) Darstellung der Ergebnisse in einem modalen Dialog.

Funktionales Konzept (UI)
Einbindung in Navigation & BottomNavigationBar:
Der Screen ist über die permanente `BottomNavigationBar` erreichbar. Beim Wechsel auf den Tab Berichte wird dieser im *selected*-Zustand farblich hervorgehoben, während die anderen Tabs dezenter dargestellt werden. Da der Kalender die Basis-Ansicht bildet, verfügt der Berichte-Screen oben links über einen Zurück-Pfeil (`IconButton`), um zum `MainScreen` zurückzukehren.
Layout-Struktur:
Der Screen verwendet ein `Scaffold`-Layout (BottomNavigationBar + Content). Im Content-Bereich wird eine leicht links ausgerichtete Überschrift Berichte angezeigt. Der Hauptbereich widmet sich der Zeitraum-Konfiguration.
Zeitraumauswahl:

- **Preset-Auswahl:**
  Unter dem Hinweistext Bitte wählen Sie einen Zeitraum für die Berichte befindet sich eine horizontale `Row` mit vier `OutlinedButton`-Komponenten: Tag, Woche, Monat, Jahr.

- **Benutzerdefinierter Zeitraum:**
  Nach dem Text oder wählen Sie den Zeitraum selber aus werden zwei klickbare Eingabeelemente dargestellt (beschriftet mit von ... und bis ...), die jeweils einen nativen Material 3 `DatePickerDialog` öffnen.

Die Aktion Suchen ist erst aktiv, wenn ein gültiger Zeitraum definiert wurde (Preset gewählt *oder* manueller Zeitraum mit `von` $\leq$ `bis`).
Aktion & Ergebnisdarstellung (Dialog):
Ein Klick auf den gefüllten Suchen-`Button` startet die Berichtserstellung. Die Ergebnisse werden in einem modalen `Dialog` (Overlay) angezeigt, um den Kontext der Auswahl nicht zu verlieren. Der Dialog ist in einer `Card` mit abgerundeten Ecken eingebettet und enthält:

1. Die Anzahl der erledigten Aufgaben.

2. Die Anzahl der nicht erledigten Aufgaben.

3. Die Summe der in diesem Zeitraum gesammelten XP.

Oben links befindet sich ein `IconButton` mit X (`Icons.Default.Close`) zum Schließen.

Fehler- und Sonderfälle (UI-Handling):
Die Fehlerbehandlung kapselt technische Details über den `UiErrorType`:

- **Validierung:** Bei ungültigem Zeitraum (fehlendes Datum oder `von` > `bis`) wird keine Suche ausgelöst (*UiErrorType:* `Validation`). Die UI zeigt eine Inline-Meldung und deaktiviert den Suchen-Button.
- **Netzwerkfehler:** Schlägt die Berechnung fehl (Timeout, Serverfehler), wird eine Fehlermeldung mit Retry-Option angezeigt, ohne die gewählte Zeitspanne zu verlieren.
- **Leere Ergebnisse:** Existieren im gewählten Zeitraum keine Aufgaben, wird der Dialog dennoch angezeigt (Werte sind 0).

Entwurfsentscheidungen:

- Ergebnisse im Dialog statt separatem Screen, um den Kontext beizubehalten.
- Client-seitige Validierung vor dem Request zur Vermeidung unnötiger Last.

Technischer Klassentwurf

Der `BerichteScreen` setzt die funktionale Beschreibung technisch um. Er nutzt die hierarchische Navigationsstruktur des MainScreens und kapselt die Ergebnisanzeige in einem Overlay.

1. Zustands-Definitionen und Typen

Enum: *`DateRangePreset`*

Definiert die Auswahlmöglichkeiten für den Zeitraum.

- *Werte:*
  - `DAY, WEEK, MONTH, YEAR`: Die Standard-Presets.
  - `CUSTOM`: Wird gesetzt, wenn der Nutzer die von/bis-Felder nutzt.

Enum: `UiErrorType`
(Referenziert aus *UI Common*) Kategorisiert Fehler wie `VALIDATION`, `NETWORK_UNAVAILABLE`, `TIMEOUT`, `SERVER_ERROR`.

2. Datenmodelle (View Data)

Klasse: `ReportResultUi`
Hält die berechneten Statistik-Daten für den Ergebnis-Dialog.

- **Attribute:**
  - `completedCount: Int` – Anzahl erledigter Aufgaben.
  - `openCount: Int` – Anzahl offener Aufgaben.
  - `totalXp: Int` – Summe der gesammelten XP im Zeitraum.

Klasse: `BerichteScreenState`
Der vollständige Zustand des Screens.

- **Attribute:**
  - `selectedPreset: DateRangePreset` – Aktuell gewählter Filter (Default: DAY).
  - `customFrom: LocalDate?` – Startdatum (nur relevant bei CUSTOM).
  - `customTo: LocalDate?` – Enddatum (nur relevant bei CUSTOM).
  - `isSearchEnabled: Boolean` – Steuert, ob der Suchen-Button klickbar ist (Validierungsstatus).
  - `isLoading: Boolean` – Zeigt an, ob der Bericht gerade berechnet wird.
  - `reportResult: ReportResultUi?` – Das Ergebnis (wenn vorhanden).
  - `showResultDialog: Boolean` – Steuert die Sichtbarkeit des Ergebnis-Overlays.
  - `activeError: UiErrorType?` – Falls gesetzt, wird eine Fehlermeldung angezeigt.

3. UI-Komponenten (View Classes)

Klasse: `BerichteScreenView`
Die Hauptansicht, die über die BottomBar geladen wird.

- **Input:** `state: BerichteScreenState`
- **Struktur (Scaffold):**
  - **TopBar:** Nutzt `BackNavigationTopBar` (Titel: Berichte, Zurück-Pfeil).
  - **Content:** Beinhaltet die `DateSelectionSection` und den Suchen-Button.
  - **Overlay:** Zeigt bedingt den `ReportResultDialog` (wenn `showResultDialog == true`).

Klasse: `DateSelectionSection`
Der Eingabebereich für den Zeitraum.

- **Elemente:**
  - **Presets:** Eine `Row` aus 4 `OutlinedButton`-Elementen.
  - **Custom:** Zwei klickbare Felder (von, bis), die den nativen `DatePickerDialog` öffnen.
- **Validierung:** Visualisiert Fehler (z.B. rote Umrandung), wenn `from > to`.

Klasse: `ReportResultDialog` (Overlay)
Der modale Dialog zur Anzeige der Statistik.

- **Input:** `data: ReportResultUi`
- **Layout:** Eine `Card` mit abgerundeten Ecken.
- **Inhalt:**
  - Zeigt die drei Kennzahlen (Erledigt, Offen, XP).
  - Enthält oben links einen `CloseIconButton` (X).

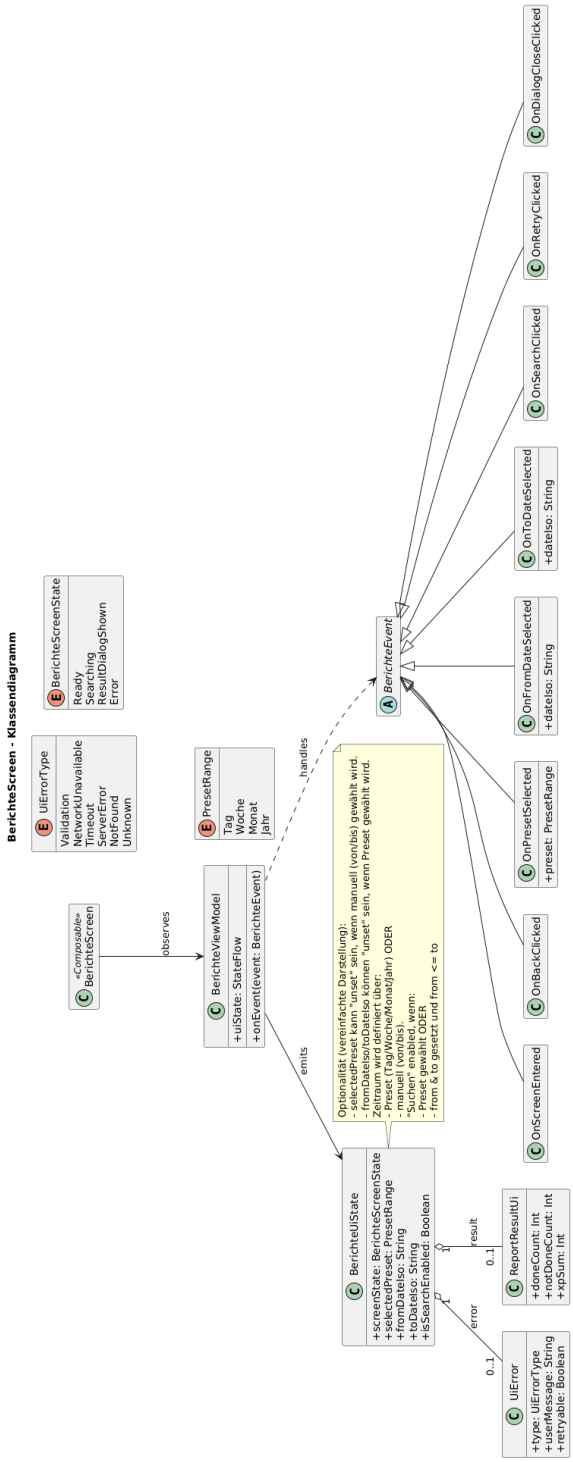
4. Schnittstellen (Interaction Listener)
Interface: `BerichteInteractionListener`

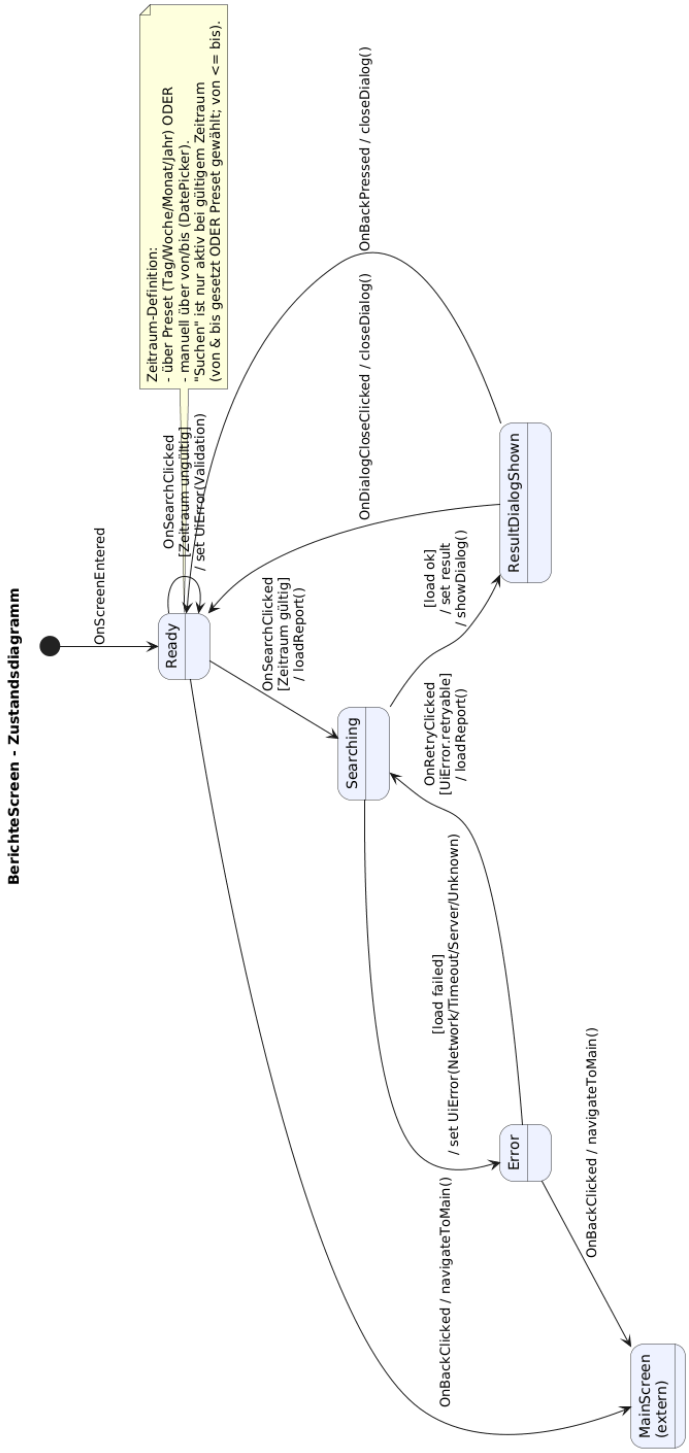
- **Navigation & Dialoge:**
  - `onBackClicked()`: Navigation zurück zum MainScreen.

  - `onCloseDialogClicked()`: Schließt den Ergebnis-Dialog.

- **Zeitraum-Auswahl:**
  - `onPresetSelected(preset: DateRangePreset)`: Wählt Tag/Woche/etc.
  - `onCustomDateClicked(isStart: Boolean)`: Öffnet den DatePicker für Start oder Ende.
  - `onDatePicked(date: LocalDate, isStart: Boolean)`: Übernimmt das Datum aus dem Picker.

- **Aktion:**
  - `onSearchClicked()`: Startet die Berichtserstellung (nur wenn `isSearchEnabled`).
  - `onRetryClicked()`: Wiederholt die Anfrage bei Netzwerkfehlern.


BerichteScreen - Klassendiagramm
«Composable»
BerichteScreen
observes
BerichteViewModel
+uiState: StateFlow
+onEvent(event: BerichteEvent)
emits
handles
UiErrorType
Validation
NetworkUnavailable
Timeout
ServerError
NotFound
Unknown
BerichteScreenState
Ready
Searching
ResultDialogShown
Error
PresetRange
Tag
Woche
Monat
Jahr
Optionalität (vereinfachte Darstellung):
- selectedPreset kann "unset" sein, wenn manuell (von/bis) gewählt wird.
- fromDateIso/toDateIso können "unset" sein, wenn Preset gewählt wird.
Zeitraum wird definiert über:
- Preset (Tag/Woche/Monat/Jahr) ODER
- manuell (von/bis).
"Suchen" enabled, wenn:
- Preset gewählt ODER
- from & to gesetzt und from <= to
BerichteUiState
+screenState: BerichteScreenState
+selectedPreset: PresetRange
+fromDateIso: String
+toDateIso: String
+isSearchEnabled: Boolean
error
0..1
result
0..1
UiError
+type: UiErrorType
+userMessage: String
+retryable: Boolean
ReportResultUi
+doneCount: Int
+notDoneCount: Int
+xpSum: Int
BerichteEvent
OnScreenEntered
OnBackClicked
OnPresetSelected
+preset: PresetRange
OnFromDateSelected
+dateIso: String
OnToDateSelected
+dateIso: String
OnSearchClicked
OnRetryClicked
OnDialogCloseClicked


BerichteScreen - Zustandsdiagramm
OnScreenEntered
Ready
OnSearchClicked
[Zeitraum ungültig]
/ set UiError(Validation)
Zeitraum-Definition:
- über Preset (Tag/Woche/Monat/Jahr) ODER
- manuell über von/bis (DatePicker).
"Suchen" ist nur aktiv bei gültigem Zeitraum
(von & bis gesetzt ODER Preset gewählt; von <= bis).
OnSearchClicked
[Zeitraum gültig]
/ loadReport()
Searching
[load ok]
/ set result
/ showDialog()
ResultDialogShown
OnDialogCloseClicked / closeDialog()
OnBackPressed / closeDialog()
OnRetryClicked
[UiError.retryable]
/ loadReport()
[load failed]
/ set UiError(Network/Timeout/Server/Unknown)
Error
OnBackClicked / navigateToMain()
OnBackClicked / navigateToMain()
MainScreen
(extern)

3.3.1.5 ProfileScreen (Realisierung von F150, F30, F40) Dieser Screen dient als zentrale Verwaltungsstelle für das Benutzerkonto und die App-Einstellungen. Das Design integriert die Konfiguration (Settings) direkt in das Profil, um die Navigation zu vereinfachen.

Funktionales Konzept (UI)
Layout-Struktur:
Die Elemente sind vertikal in einer `Column` angeordnet. Der obere Bereich zeigt die Identität des Nutzers (aus OpenID Connect), der untere Bereich listet alle verfügbaren Aktionen als Buttons auf.
Wichtige UI-Komponenten:

- **Profilbild (Avatar):** Das Nutzerbild wird dynamisch über Coil mittels `AsyncImage` geladen (URL aus OpenID Token) und als `CircleShape` dargestellt. Falls das Laden fehlschlägt, wird ein Placeholder-Avatar angezeigt.
- **Benutzername:** Unterhalb des Avatars werden Vor- und Nachname des Nutzers angezeigt (Daten aus OpenID Connect).
- **Aktions-Buttons (vertikale Liste):**
  1. **Level einsehen (F150):** Ein `OutlinedButton` öffnet einen modalen Dialog.
     - *Schließen:* `IconButton` mit X-Icon schließt den Dialog und kehrt zum Profil zurück.
     - *Inhalt:* Darstellung analog zur Rangliste mit Level-Sternen (`Icons.Filled.Star`) und einem XP-Fortschrittsbalken.
  2. **Einstellungen (Settings):** Navigiert per Push-Navigation zur Settings-Ansicht
  3. **Logout (F30):** Öffnet einen Bestätigungsdialog (`AlertDialog`). Erst nach expliziter Bestätigung werden lokale Tokens gelöscht. Zusätzlich wird ein ggf. registrierter Push-Token serverseitig *best effort* abgemeldet (Logout wird bei fehlender Verbindung nicht blockiert). Danach wird zur Login-Ansicht navigiert und der Back-Stack geleert, um eine Rücknavigation in authentifizierte Screens zu verhindern.
  4. **Account löschen (F40):** Ein farblich hervorgehobener Warn-Button. Analog zum Logout wird zwingend ein Bestätigungsdialog angezeigt. Erst nach Bestätigung wird der Löschbefehl an den Server gesendet.

Bei Erfolg werden lokale Tokens gelöscht und (wie beim Logout) zur Login-Ansicht navigiert (Back-Stack wird geleert).

Fehler- und Sonderfälle (UI-Handling):
Die Fehlerbehandlung folgt dem zentralen `UiError`-Konzept (`UiErrorType`).

- **Offline-Modus:** Schreibende Server-Aktionen sind offline deaktiviert. Insbesondere ist Account löschen nur online möglich. *UI-Reaktion:* Button deaktiviert + kurzer Hinweis (z.B. Snackbar Internet erforderlich).
- **Account löschen scheitert:** *UiErrorType:* `NetworkUnavailable`, `Timeout`, `ServerError`, `Unknown`.
  UI-Reaktion: Fehlermeldung + Retry-Möglichkeit, ohne den Nutzerkontext zu verlieren.
- **Logout (best effort Push-Abmeldung):** Das Abmelden eines Push-Tokens vom Server kann fehlschlagen (z.B. offline), blockiert den Logout jedoch nicht.

Entwurfsentscheidungen (UI):

- Settings als Sub-Screen innerhalb des Profilbereichs zur Reduktion von Navigationstiefe.
- Sicherheitsrelevante Aktionen (Logout, Account löschen) immer mit Bestätigungsdialog.
- Navigation nach Logout/Account-Löschung mit geleertem Back-Stack zur Vermeidung von Zugriff über Zurück-Navigation.

Technischer Klassentwurf

Der `ProfileScreen` fungiert als zentraler Hub für Benutzerdaten und sicherheitskritische Aktionen. Er integriert die Statusanzeige (Level) und die Navigation zu den Einstellungen.

1. Zustands-Steuerung (Enums)

Enum: *`ProfileDialogType`*

Steuert die Sichtbarkeit der modalen Dialoge, die im Profil-Screen benötigt werden.

- *Werte:*
  - `NONE`: Standardzustand, kein Dialog sichtbar.
  - `LEVEL_INFO`: Zeigt den Gamification-Status (Sterne/XP) an (F150).
  - `LOGOUT_CONFIRM`: Sicherheitsabfrage vor dem Logout (F30).
  - `DELETE_ACCOUNT_CONFIRM`: Warn-Dialog vor der Kontolöschung (F40).

2. Datenmodelle (View Data)

Klasse: `UserProfileUi`

Ein Datencontainer (DTO) für die Anzeige der Nutzeridentität.

- **Attribute:**
  - `displayName: String` – Der vollständige Name aus OpenID Connect.
  - `avatarUrl: String?` – URL zum Profilbild (Nullable, Fallback auf Platzhalter).
  - `currentLevel: Int` – Das aktuelle Level des Nutzers.
  - `currentXp: Int` – Die gesammelten Erfahrungspunkte.
  - `xpForNextLevel: Int` – Zielwert für die Fortschrittsbalken-Berechnung.

Klasse: `ProfileScreenState`

Hält den gesamten UI-Zustand des Screens.

- **Attribute:**
  - `userProfile: UserProfileUi?` – Die geladenen Nutzerdaten.
  - `activeDialog: ProfileDialogType` – Bestimmt, welches Overlay angezeigt wird.
  - `isOffline: Boolean` – Netzwerkstatus. Wenn `true`, wird der Button Account löschen deaktiviert.
  - `isProcessing: Boolean` – Zeigt einen Ladeindikator während Logout- oder Lösch-Operationen.
  - `error: UiErrorType?` – Speichert Fehlerzustände (z.B. bei fehlgeschlagener Löschung).

3. UI-Komponenten (View Classes)

Klasse: `ProfileScreenView`

Die Hauptansicht, die über die BottomNavigationBar erreicht wird.

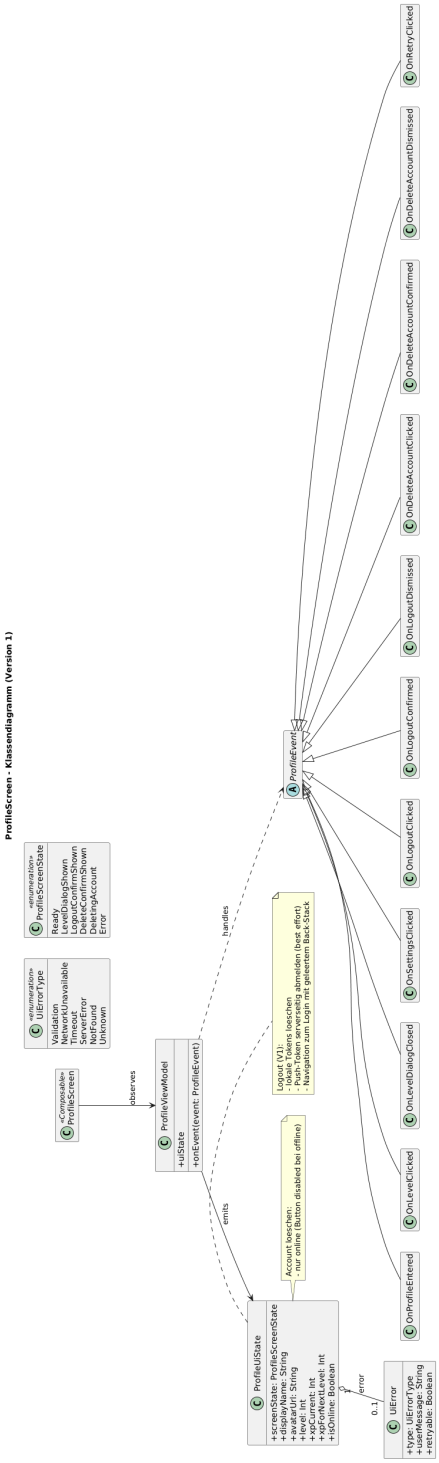
- **Input:** `state: ProfileScreenState`
- **Struktur:**
  - **Header:** Zeigt Avatar (AsyncImage) und Benutzernamen zentriert an.
  - **ActionList:** Eine vertikale Liste (`Column`) von Buttons für Level, Settings, Logout und Löschen.
  - **Overlay-Manager:** Rendert basierend auf `activeDialog` den entsprechenden `AlertDialog` oder Info-Dialog.

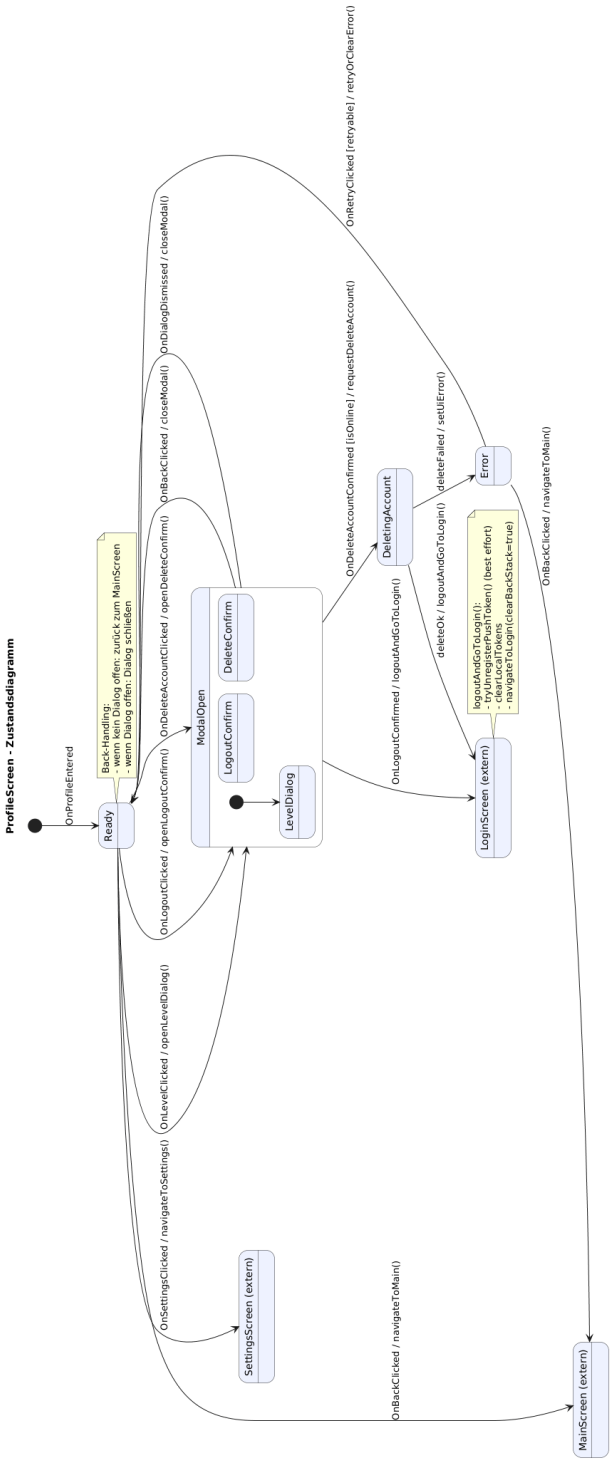
4. Schnittstellen (Interaction Listener)

Interface: *`ProfileInteractionListener`*

Definiert die Benutzerinteraktionen für das ViewModel.

- **Navigation & Info:**
  - `onShowLevelClicked()`: Öffnet den Level-Dialog.
  - `onSettingsClicked()`: Navigiert zum `SettingsScreen`.
  - `onCloseDialog()`: Schließt das aktive Overlay.
- **Konto-Aktionen:**
  - `onRequestLogout()`: Öffnet den Logout-Bestätigungsdialog.
  - `onConfirmLogout()`: Führt den Logout aus (Token-Löschung).
  - `onRequestDelete()`: Öffnet den Lösch-Dialog (nur im Online-Modus).
  - `onConfirmDelete()`: Sendet den Löschbefehl an den Server.


ProfileScreen - Klassendiagramm (Version 1)
«Composable»
ProfileScreen
observes
ProfileViewModel
+uiState
+onEvent(event: ProfileEvent)
emits
handles
«enumeration»
UIErrorType
Validation
NetworkUnavailable
Timeout
ServerError
NetworkLost
Unknown
«enumeration»
ProfileScreenState
Ready
LeaveDialogShown
LogoutConfirmShown
DeleteConfirmShown
DeleteAccount
Error
ProfileUiState
+screenState: ProfileScreenState
+displayName: String
+avatarUrl: String
+hasLevel: Bool
+xpCurrent: Int
+xpForNextLevel: Int
+isOnline: Boolean
0..1
error
UIError
+type: UIErrorType
+userMessage: String
+canRetry: Boolean
Account löschen:
- nur online (Button disabled bei offline)
Logout (V1):
- lokale Sessions löschen
- Push-Token serverseitig abmelden (best effort)
- Navigation zum Login mit geleertem Back-Stack
ProfileEvent
OnProfileEntered
OnLevelClicked
OnLevelDialogClosed
OnSettingsClicked
OnLogoutClicked
OnLogoutConfirmed
OnLogoutDismissed
OnDeleteAccountClicked
OnDeleteAccountConfirmed
OnDeleteAccountDismissed
OnRetryClicked


ProfileScreen - Zustandsdiagramm
OnProfileEntered
Ready
Back-Handling:
- wenn kein Dialog offen: zurück zum MainScreen
- wenn Dialog offen: Dialog schließen
OnLogoutClicked / openLogoutConfirm()
OnDeleteAccountClicked / openDeleteConfirm()
OnBackClicked / closeModal()
OnDialogDismissed / closeModal()
OnLevelClicked / openLevelDialog()
OnSettingsClicked / navigateToSettings()
ModalOpen
LogoutConfirm
DeleteConfirm
LevelDialog
OnDeleteAccountConfirmed [isOnline] / requestDeleteAccount()
OnLogoutConfirmed / logoutAndGoToLogin()
OnRetryClicked [retryable] / retryOrClearError()
DeletingAccount
deleteOk / logoutAndGoToLogin()
deleteFailed / setUIError()
Error
logoutAndGoToLogin():
- tryUnregisterPushToken() (best effort)
- clearLocalTokens
- navigateToLogin(clearBackStack=true)
LoginScreen (extern)
OnBackClicked / navigateToMain()
SettingsScreen (extern)
OnBackClicked / navigateToMain()
MainScreen (extern)

Settings-Ansicht (Sub-Screen)

Dieser Bereich beinhaltet lokale App-Konfigurationen. Er wird als eigenständiger Screen realisiert, der per Push-Navigation vom ProfileScreen geöffnet wird.

Navigation & Layout:

- **TopAppBar:** Titel Einstellungen und Zurück-Pfeil (Navigation Icon). Ein Klick navigiert zurück zum Profil.
- **Listenansicht:** Einstellungen sind als vertikale Liste von `ListItem`-Komponenten angeordnet.

Enthaltene Komponenten:

- **Dark Mode (Aus/An):** `Switch` (nur UI-Zustand).
- **Sprache (DE/EN):** `SegmentedButton` zur Auswahl.
- **Benachrichtigungen (Aus/An):** `Switch` für Push-Benachrichtigungen.
- **Import:** Klickbares Element Kalender importieren, öffnet den System-Dateimanager zur Auswahl einer Datei (nur Dateiauswahl in Version 1, kein Parsing/Import).

Push-Benachrichtigungen (ereignisbasiert, Version 1):

Benachrichtigungen werden ausschließlich ereignisbasiert verwendet (z.B. Global-Challenge-Updates, Gruppeneinladungen, neue Aufgaben). Zeitgesteuerte Reminder sind in Version 1 nicht Bestandteil des Scopes, um die Komplexität zu begrenzen. Auf Android 13+ wird beim Aktivieren der Option Benachrichtigungen die Berechtigung `POST_NOTIFICATIONS` angefragt; bei Ablehnung wird der Toggle zurückgesetzt und ein Hinweis angezeigt. Die Einstellung ist in Version 1 **nicht persistent** und gilt nur für die aktuelle App-Sitzung (nach einem Neustart ist der Default-Zustand Aus).

Fehler- und Sonderfälle (UI-Handling):

- **Import abgebrochen (kein Fehler):** Wenn der Nutzer den Dateimanager schließt, bleibt der Screen unverändert.
- **Dateizugriff fehlgeschlagen:** *UiErrorType:* `Unknown`. *UI-Reaktion:* Hinweis (Snackbar) und Rückkehr in den Ready-Zustand.

Entwurfsentscheidungen (UI):

- Komplexitätsreduktion: Settings sind **nicht persistent** (werden nicht über App-Neustart hinweg gespeichert).

Klassentwurf: Einstellungen (Settings)

Der `SettingsScreen` ist ein Sub-Screen des Profilbereichs. Er verwaltet lokale, nicht-persistente App-Konfigurationen und wird per Push-Navigation geöffnet.

1. Typ-Definitionen

Enum: *`AppLanguage`*

Definiert die unterstützten Sprachen der Anwendung.

- *Werte:* `GERMAN` (Deutsch), `ENGLISH` (Englisch).

2. Datenmodelle (View Data)

Klasse: *`SettingsScreenState`*

Hält die lokalen Konfigurationen für die aktuelle Sitzung.

- **Attribute:**
  - `isDarkMode: Boolean` – Steuert das UI-Thema (Hell/Dunkel).
  - `selectedLanguage: AppLanguage` – Die aktuell gewählte Sprache.
  - `areNotificationsEnabled: Boolean` – Status der Push-Benachrichtigungen.
- *Hinweis:* Da die Einstellungen in Version 1 nicht persistent sind, wird dieser State beim Neustart der App auf Standardwerte zurückgesetzt.

3. UI-Komponenten (View Classes)

Klasse: *`SettingsScreenView`*

Die Ansicht für die Konfiguration.

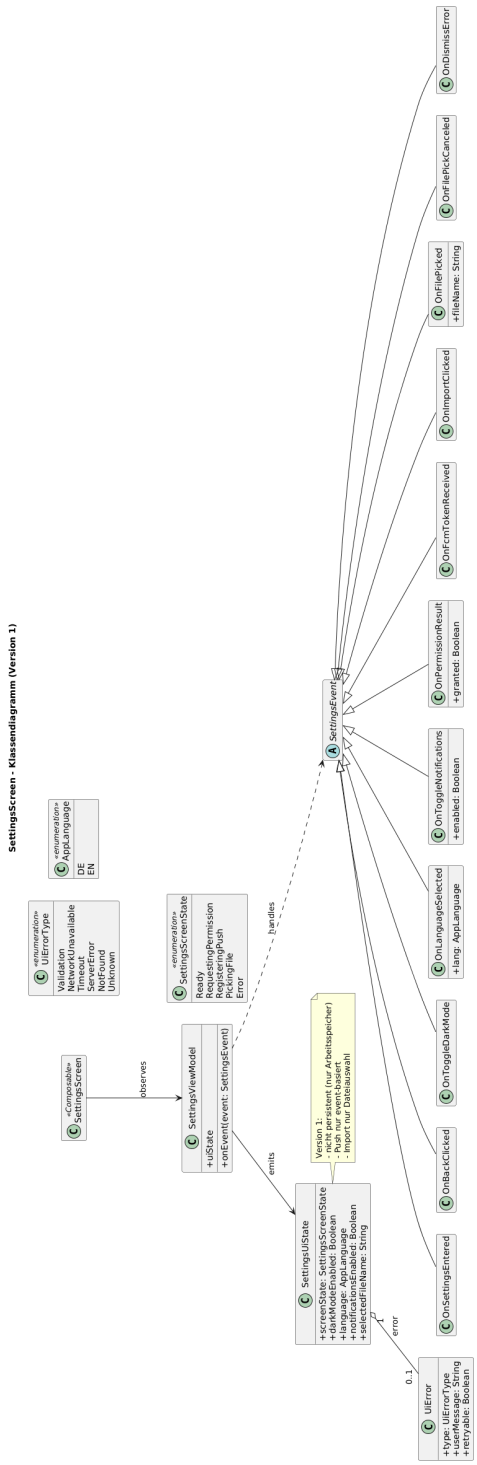
- **Input:** `state: SettingsScreenState`
- **Layout:** Ein `Scaffold` mit eigener TopBar.
- **Struktur:**
  - **TopAppBar:** Titel Einstellungen und Zurück-Pfeil (Navigation Icon).
  - **Content:** Eine vertikale Liste von `SettingsItem`-Komponenten (Switches, SegmentedButton, Clickable Rows).

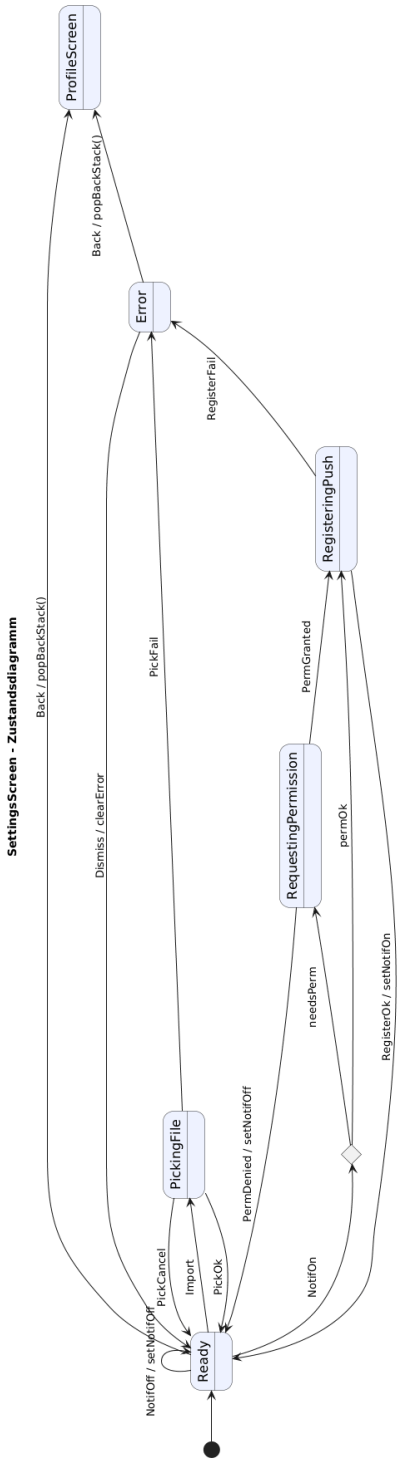
4. Schnittstellen (Interaction Listener)

Interface: `SettingsInteractionListener`

Behandelt die Konfigurationsänderungen und Navigation.

- **Navigation:**
  - `onBackClicked()`: Schließt den Screen und kehrt zum Profil zurück.
- **Konfiguration:**
  - `onToggleDarkMode(enabled: Boolean)`: Schaltet das Design um.
  - `onLanguageSelected(lang: AppLanguage)`: Ändert die Sprache der UI-Strings.
  - `onToggleNotifications(enabled: Boolean)`: Fragt (ab Android 13) die Berechtigung an und setzt den internen State.
  - `onImportCalendarClicked()`: Öffnet den System-Dateimanager zur Dateiauswahl.


SettingsScreen - Klassendiagramm (Version 1)
«Composable»
SettingsScreen
observes
SettingsViewModel
+uiState
+onEvent(event: SettingsEvent)
emits
«enumeration»
UiErrorType
Validation
NetworkUnavailable
Timeout
ServerError
NotFound
Unknown
«enumeration»
SettingsScreenState
Ready
RequestingPermission
RegisteringPush
PickingFile
Error
«enumeration»
AppLanguage
DE
EN
handles
SettingsUiState
+screenState: SettingsScreenState
+darkModeEnabled: Boolean
+language: AppLanguage
+notificationsEnabled: Boolean
+selectedFileName: String
Version 1:
- nicht persistent (nur Arbeitsspeicher)
- Pull per event-based
- Import nur Dateiauswahl
error
0..1
UiError
+type: UiErrorType
+userMessage: String
+retryable: Boolean
SettingsEvent
OnSettingsEntered
OnBackClicked
OnToggleDarkMode
OnLanguageSelected
+lang: AppLanguage
OnToggleNotifications
+enabled: Boolean
OnPermissionResult
+granted: Boolean
OnFcmTokenReceived
OnImportClicked
OnFilePicked
+fileName: String
OnFilePickCanceled
OnDismissError


SettingsScreen - Zustandsdiagramm
ProfileScreen
Back / popBackStack()
Back / popBackStack()
Error
RegisterFail
RegisteringPush
PickFail
PermGranted
Dismiss / clearError
RequestingPermission
permOk
needsPerm
RegisterOk / setNotifOn
PickingFile
PermDenied / setNotifOff
PickCancel
Import
PickOk
NotifOn
NotifOff / setNotifOff
Ready

3.3.2 ViewModel-Schicht

Die ViewModel-Schicht stellt den Zustand der Benutzeroberfläche bereit und kapselt die Interaktionslogik gemäß dem MVVM-Muster. Die ViewModels sind nach dem Unidirectional Data Flow Prinzip aufgebaut. Jedes ViewModel hält den Zustand für genau einen Screen in einem StateFlow und verarbeitet Interaktionen als Methodenaufrufe. Es folgt eine Beschreibung der einzelnen ViewModel Klassen.

3.3.2.1 View und ViewModel Umsetzung Die Benutzeroberfläche (View-Komponente) wird mithilfe der Jetpack Compose Library deklarativ beschrieben. Ein "Composable" ist dabei eine Kotlin-Funktion, die UI-Elemente erzeugt. Für jeden der fünf Haupt-Screens der App (Login, Kalender, Rangliste, Berichte, Profil) existiert eine sogenannte Screen-Level" Composable, die den gesamten Bildschirminhalt definiert.

Die ViewModel-Komponente folgt der 1-zu-1-Zuordnung: Zu jedem Screen gehört genau eine ViewModel-Klasse, die den Zustand dieses Screens verwaltet und überdauert, auch wenn der Screen neu gezeichnet (recomposed) wird.

3.3.2.2 View-ViewModel Schnittstelle (Unidirectional Data Flow) Die Kommunikation zwischen View und ViewModel folgt dem Muster des "Unidirectional Data Flow":

- **State (Zustand):** Das ViewModel stellt den aktuellen UI-Zustand (z.B. `LoginUiState`) als beobachtbares Datenobjekt bereit (mittels StateFlow).
- **Beobachtung:** Die Composable-Funktion beobachtet diesen Zustand (via `observeAsState()`). Sobald das ViewModel einen neuen Zustand sendet (z.B.`isLoading = true`), wird die UI von Jetpack Compose automatisch und effizient neu gezeichnet.
- **Events (Ereignisse):** Interaktionen des Nutzers (z.B. Klick auf "Speichern") werden von der View als Methodenaufruf an das ViewModel weitergegeben (z.B.`viewModel.onSaveClicked()`). Das ViewModel verarbeitet die Logik und aktualisiert anschließend den State.

3.3.2.3 LoginViewModel Dieses ViewModel steuert den gesamten Anmelde- und Registrierungsprozess. Es verwaltet den Zustand (`LoginUiState`), der neben dem aktuellen Ladeindikator (`isLoading`) auch speichert, ob der Nutzer bereits erfolgreich authentifiziert wurde (`isAuthenticated`). Ein zentraler Aspekt ist der Onboarding-Modus: Falls ein Nutzer sich neu via OpenID anmeldet, setzt das ViewModel das Flag `isUsernameRequired` auf wahr und fordert zur Eingabe eines Benutzernamens auf. Der Nutzer tippt diesen in das Textfeld ein, was über `onUsernameChanged(text:String)` den Zustand aktualisiert. Mit `onRegisterUsernameClicked()` wird der gewählte Name final an Model gesendet, um die Registrierung abzuschließen. Der reguläre Login wird über `onLoginClicked()` initiiert.

- **Datenstrukturen (State):** `uiState: StateFlow<LoginUiState>`
  - `isLoading` (Boolean): Steuert Ladeanzeigen.
  - `isAuthenticated` (Boolean): Signalisiert erfolgreichen Login (Navigation zum MainScreen).
  - `isUsernameRequired` (Boolean): `true`, wenn Account neu ist (Onboarding-Modus).
  - `usernameInput` (String): Text im Eingabefeld.
  - `errorMessage` (String?): Validierungsfehler.
- **Methoden (Events):**
  - `onLoginClicked()`: Startet OAuth/OpenID Flow.
  - `onUsernameChanged(text:String)`: Aktualisiert den Input-State.
  - `onRegisterUsernameClicked()`: Validiert den Namen und sendet ihn an das Model.

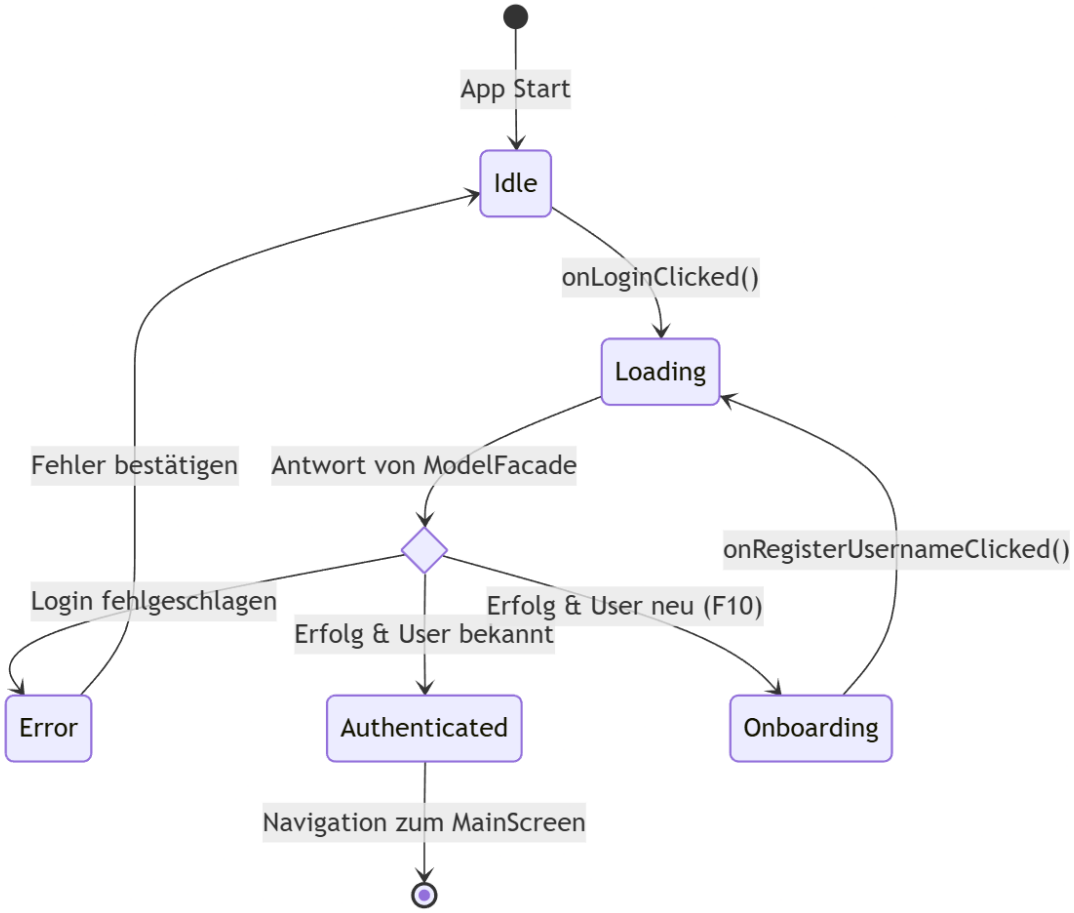


Abbildung 3.8: Zustandsdiagramm: Innere Logik des LoginViewModels.

3.3.2.4 LeaderboardViewModel Dieses ViewModel ist für die Darstellung der globalen Rangliste zuständig. Es hält den Zustand `LeaderboardUiState`, der eine Liste von bereits aufbereiteten Einträgen (`entries`) enthält. Jeder Eintrag in dieser Liste kapselt dabei alle nötigen UI-Informationen wie den formatierten Rang, den Namen und den Level, der die Anzahl der anzuzeigenden Stern-Icons bestimmt. Um die Liste auf den neuesten Stand zu bringen, kann der Nutzer die Methode `refreshLeaderboard()` aufrufen (z. B. durch eine Pull-to-Refresh Geste), woraufhin das ViewModel die aktuellen Daten asynchron vom Server abruft und den Ladezustand entsprechend setzt.

- **Datenstrukturen (State):** `uiState: StateFlow<LeaderboardUiState>`
  - `isLoading` (Boolean): Ladeindikator.
  - `entries` (List`<LeaderboardEntry>`): Liste der Einträge.
  - *Entry-Format:* Enthält `rank` (String), `level` (Int für Sterne), `xpProgress` (Float) und `xpLabel` (String).
- **Methoden (Events):**
  - `refreshLeaderboard()`: Ruft `ModelFacade.globalLeaderboard()` auf Das ViewModel berechnet anschließend aus den empfangenen XP-Werten das Level und den relativen Fortschritt (0.0 - 1.0) für die UI-Anzeige.

3.3.2.5 ReportViewModel Dieses ViewModel verwaltet die Erstellung von Berichten und Statistik-Auswertungen. Es speichert im `ReportUiState` den aktuell vom Nutzer gewählten Zeitraum (z. B. Woche oder Monat) sowie die Sichtbarkeit des Ergebnis-Dialogs. Wählt der Nutzer über die View einen Filterzeitraum aus, wird dies durch `onFilterSelected()` verarbeitet. Ein Klick auf die "Suchen"-Schaltfläche löst die Methode `onSearchClicked()` aus. Das ViewModel fragt daraufhin die ReportData (erledigte vs. offene Tasks, gesammelte XP) beim Model ab und öffnet bei Erfolg das Ergebnisfenster (`isDialogVisible = true`), welches über `onDialogClose()` wieder geschlossen werden kann.

- **Datenstrukturen (State):** `uiState: StateFlow<ReportUiState>`
  - `selectedPeriod` (Enum): Tag, Woche, Monat, Jahr, Custom.
  - `isDialogVisible` (Boolean): Steuert Sichtbarkeit des Popups.

  - `reportData` (ReportData): Ergebnisse (Erledigte Tasks, XP).

- **Methoden (Events):**
  - `onFilterSelected(preset:DateRangePreset)`: Setzt das Zeitfenster im State.
  - `onSearchClicked()`: Startet DB-Abfrage (F140) und öffnet Dialog.
  - `onDialogClose()`: Schließt das Fenster.

3.3.2.6 ProfileViewModel Dieses ViewModel fungiert als zentrale Steuerung für das Nutzerprofil und globale App-Einstellungen. Der Zustand `ProfileUiState` bündelt die persönlichen Daten des Nutzers (Anzeigename, Bild) sowie den aktuellen Gamification-Fortschritt (Level, XP-Balken). Zudem steuert das ViewModel die Logik für kritische Dialoge: Über Methoden wie `onLogoutClicked()` oder `onDeleteAccountClicked()` werden Sicherheitsabfragen eingeblendet. Erst wenn der Nutzer diese bestätigt (z. B. via `onLogoutConfirmed()`), führt das ViewModel die eigentliche Abmeldung oder Löschung durch. Auch lokale Einstellungen wie der Dunkelmodus können hier über `toggleDarkMode()` direkt umgeschaltet werden.

- **Datenstrukturen (State):** `uiState: StateFlow<ProfileUiState>`
  - `userData`: Name und Bild-URL (werden direkt aus dem OpenID-Token extrahiert, da sie nicht Teil des User-Models sind).
  - `gamification`: Level, XP, Progress-Float (M160).
  - `dialogState`: Booleans für Level-, Logout- und Delete-Dialoge.
  - `settings`: Flags für DarkMode (W70), Sprache (W50), Notification (W10).

- **Methoden (Events):**
  - `onLevelClicked()`, `onLogoutClicked()`, `onDeleteAccountClicked()`: Öffnen Dialoge.
  - `onLogoutConfirmed()`- Ruft `ModelFacade.logout()` auf und leitet anschließend zur Login-View weiter. `onDeleteConfirmed()`-Ruft `ModelFacade.deleteAccount()` auf. Bei Erfolg werden alle lokalen Daten gelöscht und zur Login-View navigiert.(F30, F40).
  - `toggleDarkMode()`, `setLanguage(code)`: Ändern Einstellungen.

3.3.2.7 MainViewModel (Kalender & Editor) Dieses ViewModel bildet das Herzstück der Anwendung und steuert die Kalenderansicht. Es verwaltet den umfangreichen `MainUiState`, der das aktuell fokussierte Datum, den Ansichtsmodus (Tag/Woche/Monat) und die – basierend auf den aktiven Gruppenfiltern – gefilterte Liste der Termine enthält. Nutzerinteraktionen zur Navigation werden über `onDateSwipe()` verarbeitet, um im Kalender zu blättern. Das ViewModel koordiniert zudem den Editor: Über `onFabClicked()` wird der Erstell-Modus für neue Events aktiviert, während `onEventClicked()` ein existierendes Event zum Bearbeiten in das BottomSheet lädt. Aufgaben können direkt in der Liste über `onTodoChecked()` als erledigt markiert werden, woraufhin das ViewModel die Statusänderung an die Model-Schicht weiterleitet.

- **Datenstrukturen (State):** `uiState: StateFlow<MainUiState>`
  - `viewMode` (Enum): Ansicht (Tag/Woche/Monat) (M140).
  - `currentDate` (LocalDate): Fokus-Datum.
  - `events` (List<`Event`>): Gefilterte Liste.
  - `selectedEvent` (Event?): `null` = Erstellen-Modus, sonst Bearbeiten.
  - `selectedGroupIds` (Set<`Int`>): Aktive Filter.
  - `isGroupCreationDialogOpen` (Boolean): Steuert Gruppen-Popup.
- **Methoden (Events):**
  - **Navigation:** `onDateSwipe(dir)`, `onDateSelected(date)`.
  - **Filter:** `toggleGroupFilter(id)`.
  - **Actions:** `onEventClicked(event)`, `onFabClicked()`, `onTodoChecked(event, done)`, `onDeleteConfirmed()`.
  - **Groups:** `onCreateNewGroup(name, color)` Ruft `ModelFacade.createGroup(name, color)` auf.

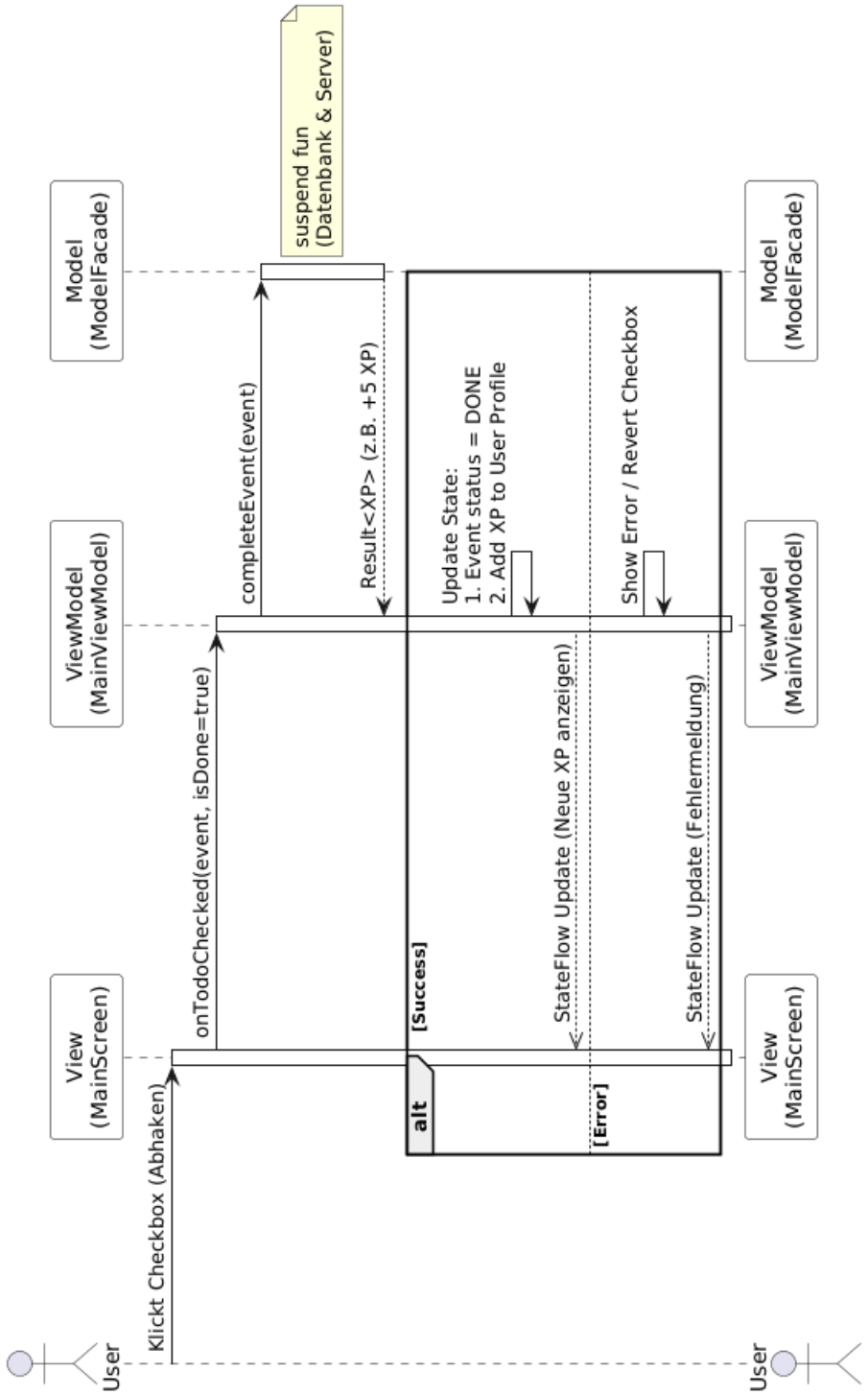


Abbildung 3.9: Sequenzdiagramm: Ablauf beim Erledigen eines Events. Zeigt den asynchronen Aufruf an die ModelFacade.

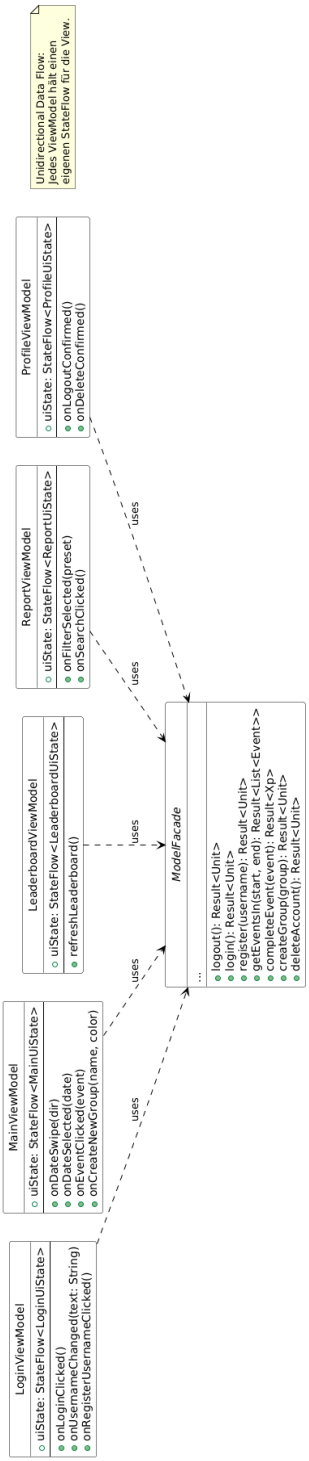


Abbildung 3.10: Klassendiagramm der ViewModel-Schicht mit zentraler ModelFacade. Alle ViewModels nutzen die Facade für Datenzugriffe.

3.3.3 Model

3.3.3.1 Model Facade Für die Kommunikation zwischen ViewModel und Model stellt das Model die einheitliche Schnittstelle `ModelFacade` bereit. Zur Durchführung asynchroner Netzwerk- und Datenbankoperationen werden `suspend`-Funktionen verwendet.

Im Folgenden sind die Methoden des Interfaces im Detail aufgeführt:

- **fun personalAccount(): StateFlow<User>**
  Liefert jederzeit den aktuellen Account des angemeldeten Nutzers. Um Änderungen am Level- und XP-Wert unmittelbar darzustellen, wird – analog zur lokalen Datenspeicherung in Room – ein `StateFlow` verwendet.
- **suspend fun friends(): Result<List<User>>**
  Gibt eine Liste aller Benutzer zurück, die mit dem angemeldeten Nutzer befreundet sind.
- **suspend fun login(): Result<Unit>**
  Führt den Login-Prozess unter Verwendung des lokal gespeicherten Authentifizierungs-Tokens durch.
- **suspend fun register(username: String): Result<Unit>**
  Registriert einen neuen Benutzer mit dem angegebenen Benutzernamen beim Server.
- **suspend fun logout(): Result<Unit>**
  Meldet den aktuell angemeldeten Nutzer ab und beendet die Sitzung.
- **suspend fun deleteAccount(): Result<Unit>**
  Löscht den Account des Nutzers unwiderruflich sowohl aus der lokalen Datenbank als auch vom Server.
- **suspend fun getEventsIn(start: LocalDateTime, end: LocalDateTime): Result<List<Event>>**
  Liefert alle Events, deren Zeitspanne innerhalb des angegebenen Zeitraums liegt.
- **suspend fun createEvent(event: Event): Result<Unit>**
  Speichert ein neues Event sowohl lokal auf dem Client als auch serverseitig.
- **suspend fun createGroup(group: Group): Result<Unit>**
  Erstellt eine neue Gruppe und speichert diese lokal sowie auf dem Server.

- **suspend fun updateEvent(event: Event): Result<Unit>**
  Ersetzt ein bestehendes Event mit derselben Event-ID in der Datenbank durch das übergebene Objekt.

- **suspend fun completeEvent(event: Event): Result<Xp>**
  Markiert ein spezifisches Event als abgeschlossen und gibt die als Belohnung vergebenen Erfahrungspunkte zurück.

- **suspend fun deleteEvent(event: Event, deleteSeries: Boolean): Result<Unit>**
  Löscht ein spezifisches Event. Der Parameter `deleteSeries` gibt an, ob zusätzlich alle Events derselben Serie entfernt werden sollen.

- **suspend fun globalLeaderboard(): Result<List<LeaderboardEntry>>**
  Gibt eine Liste der 25 Benutzer mit den meisten Erfahrungspunkten zurück.

- **suspend fun generateReport(start: LocalDateTime, end: LocalDateTime): Result<ReportData>**
  Erzeugt für den angegebenen Zeitraum einen Bericht über Aktivität und Fortschritt in Form eines `ReportData`-Objekts.

3.3.3.2 RequestAPI Die RequestAPI Schnittstelle wird verwendet, um mit der REST-API des Servers zu kommunizieren. Das Interface stellt Methoden für alle API-Endpunkte abgesehen von login bereit. Die Response-Body-DTOs der Endpunkte werden in generischen `Response`-Typen gekapselt, welche zusätzlich den HTTP-Statuscode enthalten.

- **getActiveUser(userID: String): Response<User>** Liefert die Daten des aktuell aktiven Benutzers anhand der übergebenen Benutzer-ID.
- **updateActiveUser(user: User): Response<Unit>** Aktualisiert die Profildaten des aktuell aktiven Benutzers auf dem Server.
- **getUserGroups(userID: String): Response<Map<String, Group>>** Gibt alle Gruppen zurück, denen der angegebene Benutzer angehört, abgebildet über ihre IDs.
- **getUserEvents(userID: String): Response<Map<String, Event>>** Liefert alle persönlichen Events des Benutzers, die diesem zugeordnet sind.
- **getUserAppointments(userID: String): Response<Map<String, Appointment>>** Gibt alle Termine des Benutzers zurück, inklusive zukünftiger und vergangener Einträge.
- **addUserAppointment(userID: String, newAppointment: Appointment): Response<Unit>** Fügt einen neuen Termin für den angegebenen Benutzer hinzu.
- **updateUserAppointment(userID: String, updatedAppointment: Appointment): Response<Unit>** Aktualisiert einen bestehenden Termin des Benutzers mit neuen Informationen.
- **getUserToDo(userID: String): Response<Map<String, ToDo>>** Ruft alle To-Do-Einträge des Benutzers ab.
- **addUserToDo(newToDo: ToDo): Response<Unit>** Erstellt einen neuen To-Do-Eintrag für den Benutzer.
- **updateUserToDo(userID: String, updatedToDo: ToDo): Response<Unit>** Aktualisiert einen bestehenden To-Do-Eintrag des Benutzers.
- **getUserSharedEvents(userID: String): Response<Map<String, SharedEvent>>** Liefert alle mit dem Benutzer geteilten Events, z. B. Gruppentermine oder freigegebene Ereignisse.

- **getUserFriends(userID: String): Response<Map<String, User>>**
  Gibt die Freundesliste des Benutzers zurück, inklusive der zugehörigen Benutzerdaten.
- **removeUserFriend(userID: String, friendID: String): Response<Unit>**
  Entfernt einen Freund aus der Freundesliste des Benutzers.
- **addUserFriend(userID: String, friendID: String): Response<Unit>**
  Fügt einen Benutzer anhand seiner ID zur Freundesliste hinzu.
- **getUsers(userID: List<String>): Response<Map<String, User>>**
  Ruft die Benutzerdaten mehrerer Benutzer anhand einer Liste von Benutzer-IDs ab.

3.3.3.3 Interfaces Im Folgenden werden alle zentralen Interfaces des Modells beschrieben. Sie definieren die fachlichen Kernobjekte der Anwendung sowie deren öffentlich zugängliche Methoden.

- **User**
  - **username(): String**
    Gibt den eindeutigen Benutzernamen eines Nutzers zurück.
  - **experiencePoints(): XP**
    Gibt die Gesamtanzahl der Erfahrungspunkte zurück, die der Nutzer seit der Registrierung gesammelt hat.
- **Event**
  - **eventID(): String**
    Gibt einen eindeutigen Identifikator für dieses Event zurück.
  - **eventName(): String**
    Gibt den Namen des Events zurück.
  - **description(): String**
    Gibt eine textuelle Beschreibung des Events zurück.
  - **end(): LocalDateTime**
    Gibt den Endzeitpunkt bzw. die Deadline des Events zurück.
  - **experiencePoints(): XP**
    Gibt die für das Event vorgesehenen Erfahrungspunkte zurück. Bei ToDo-Events wird solange das `NoXP`-Objekt zurückgegeben, bis das Event abgeschlossen ist.
  - **checkCompletion(): CompletionDetail**
    Gibt Informationen über den Abschlussstatus des Events zurück.
- **Appointment** (erbt von Event)
  - **group(): Group**
    Gibt die dem Appointment zugeordnete Gruppe zurück.
  - **start(): LocalDateTime**
    Gibt den Startzeitpunkt des Appointments zurück.
- **ToDo** (erbt von Event)
  Besitzt keine zusätzlichen Methoden und erweitert das `Event`-Interface ausschließlich semantisch.

- **SharedEvent** (erbt von Appointment)
  Besitzt keine zusätzlichen Methoden gegenüber `Appointment` und dient zur Modellierung gemeinsamer Termine.

- **Group**
  - **groupID(): String**
    Gibt einen eindeutigen Identifikator der Gruppe zurück.
  - **groupName(): String**
    Gibt den Namen der Gruppe zurück.
  - **groupColour(): Color**
    Gibt die zur visuellen Darstellung verwendete Farbe der Gruppe zurück.

- **Achievement**
  - **achievementName(): String**
    Gibt den Namen des Achievements zurück.
  - **achievementDescription(): String**
    Gibt die Beschreibung des Achievements zurück.
  - **checkAchievement(): CompletionDetail**
    Gibt Informationen über den Abschlussstatus des Achievements zurück.

- **CompletionDetail**
  - **completed(): bool**
    Gibt an, ob das zugehörige Objekt abgeschlossen wurde.
  - **identifier(): String**
    Gibt den eindeutigen Identifikator des abgeschlossenen Objekts zurück.
  - **completionDate(): LocalDateTime**
    Gibt den Zeitpunkt des Abschlusses zurück.

- **XP**
  - **xpValue(): int**
    Gibt den numerischen Wert der Erfahrungspunkte zurück.

  Das `XP`-Interface abstrahiert einen einfachen Ganzzahlwert. Zusätzlich wird es vom Null-Object `NoXP` implementiert, um das explizite Fehlen von Erfahrungspunkten darzustellen.

- **Response<T>**
  - **status(): int**
    Gibt einen HTTP-ähnlichen Statuscode zur Bewertung der Anfrage zurück.
  - **response(): T?**
    Gibt optional ein Antwortobjekt des Typs `T` zurück.

3.3.3.4 Implementierung

Model Das Model-Facade-Interface wird von der Klasse `Model` implementiert. Es stellt die zentrale Zugriffsschicht für die Anwendungslogik dar. Zur Initialisierung wird ein Objekt vom Typ `RequestAPI` benötigt, welches die Kommunikation mit der Server-API kapselt.

ClientAPI Das `RequestAPI`-Interface wird von der Klasse `ClientAPI` implementiert. Diese Klasse ist für die konkrete Ausführung der REST-Anfragen verantwortlich und übernimmt die Serialisierung sowie Deserialisierung der Daten zwischen Client und Server.

PersonalUser Zur Modellierung des aktuell angemeldeten Nutzers wird die Klasse `PersonalUser` verwendet. Sie speichert zusätzlich die Anzahl der am aktuellen Tag gesammelten Erfahrungspunkte (XP). Die Klasse stellt eine Methode zur Erhöhung des XP-Werts bereit. Zur Initialisierung werden der Benutzername sowie ein `ModelData`-Objekt benötigt.

RegularUser Beschreibt einen regulären Benutzer der Anwendung, beispielsweise einen Freund des aktuellen Nutzers. Diese Klasse enthält ausschließlich allgemeine Benutzerdaten ohne zusätzliche spiel- oder statusbezogene Informationen.

LeaderboardEntry Repräsentiert einen Benutzer innerhalb der Rangliste. Zusätzlich zu den Benutzerdaten speichert diese Klasse die aktuelle Platzierung des Nutzers im Leaderboard.

UserAppointment Implementiert die `Appointment`-Schnittstelle. Die Klasse wird mit einem `ModelData`-Objekt initialisiert, welches sicherstellt, dass lokale und serverseitig gespeicherte Daten synchron bleiben. Über das `ModelData`-Objekt können Änderungen an Terminen persistent gespeichert werden. Zusätzlich stellt die Klasse die Methode `complete(): Xp` bereit, welche den Termin abschließt und die entsprechenden Erfahrungspunkte speichert.

UserToDo Implementiert die `ToDo`-Schnittstelle und wird analog zu `UserAppointment` mit einem `ModelData`-Objekt initialisiert. Sie bietet Funktionen zum Abschließen eines To-Do-Eintrags und zur Speicherung des Fortschritts.

UserSharedEvent Implementiert die `SharedEvent`-Schnittstelle. Wie auch `UserAppointment` und `UserToDo` nutzt diese Klasse ein `ModelData`-Objekt zur Synchronisation mit lokalen und serverseitigen Daten. Zusätzlich kann ein geteiltes Event abgeschlossen und gespeichert werden.

UserGroup Die Klasse `UserGroup` implementiert das `Group`-Interface und modelliert eine Benutzergruppe, beispielsweise für gemeinsame Events oder Aufgaben.

UserAchievement Die Klasse `UserAchievement` implementiert die `Achievement`-Schnittstelle und beschreibt freigeschaltete oder erreichbare Erfolge eines Benutzers.

EventCD und AppointmentCD Das `CompletionDetail`-Interface wird von den Klassen `EventCD` und `AppointmentCD` implementiert. Diese Klassen kapseln zusätzliche Abschlussinformationen zu Events beziehungsweise Terminen, wie etwa Abschlusszeitpunkt oder vergebene Erfahrungspunkte.

Abbildung 3.11: Klassendiagramm zeigt die ModelFacade und RequestAPI Schnitstellen und deren Implementierungen.

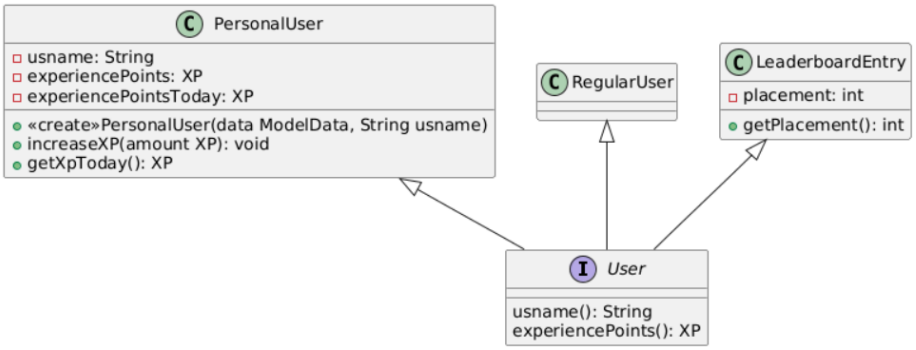


Abbildung 3.12: Klassendiagramm beshreibt alle Implementationen des User Interfaces

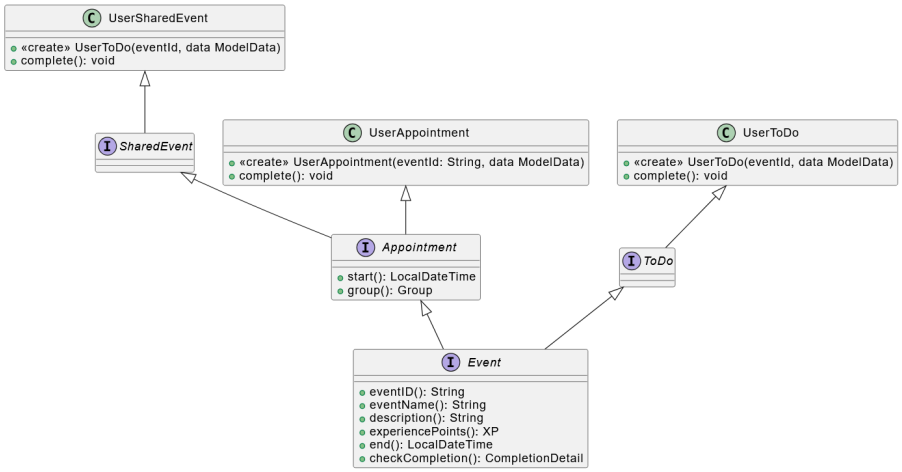


Abbildung 3.13: Klassendiagramm beschreibt die Vererbungen und Implementationen des Event interfaces.

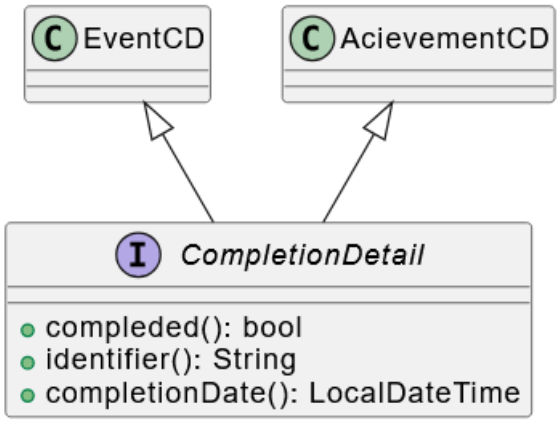


Abbildung 3.14: Klassendiagramm zeigt die Zwei implementationen von CompletionDetail. Identifier ist im fall von Events die Eventid, und für achievements der name.

3.3.3.5 Weitere Klassen

ModelData Datenbankabfragen sowie updates und löschen von Events wird im Model intern von der ModelData Klasse übernommen. ModelData Nutzt dafür selbst die Klassen UserObserver und EventObserver um die Daten über User und Events aktuell zu halten und abzufragen. Die Informatonen werden bei observe methoden direkt vom Server über die schnitstelle **RequestAPI** geholt, falls jedoch keine Verbindung möglich ist wird auf die Lokale Datenbank zugegriffen. Der Lokale Datenbank zugriff wird über das **LocalData** Object gehandhabt. Diese zwei Objekte müssen für alle drei Klassen bei der initialisierung übergeben werden.

- **UserObserver**
  - **observeCurrentUser(): StateFlow<User>**
    Gibt den aktuellen Zusatnd des Angemeldeten Nutzers als StateFlow des User objekts zurück.
  - **observeFriends(): StateFlow<List<User>>**
    Gibt den aktuellen Zusatnd aller Freunde des Angemeldeten Nutzers als StateFlow einer Liste an Usern zurück.
  - **observeLeaderboard(): StateFlow<List<User>>**
    Gibt den aktuellen Zusatnd der Nutzer mit dem meisten Erfahrungspunkten als StateFlow einer Liste an Usern zurück.
  - **refreshCurrentUser(): void**
    Holt den aktuellen Zustand des angemeldeten Users vom Server und speichert ihn in der Lokalen Datenbank.
  - **refreshFriends(): void**
    Holt den aktuellen Zustand aller Freunde des angemeldeten Users vom Server und speichert ihn in der Lokalen Datenbank.
  - **refreshLeaderboard(): void**
    Holt den aktuellen Zustand der Nutzer mit dem meisten Erfahrungspunkten vom Server und speichert ihn in der Lokalen Datenbank.
  - **updateUser(updatedUser: User): Result<Unit>**
    Ändert das User-objekt mit dem gegeben Username auf dem Server und der Lokalen datenbank. Ist keine verbindung mit dem Server Möglich gibt es eine Fehlermeldung.

- **EventObserver**
  EventObserver verwendet intern den XpDistributor, um an den richtigen Zeitpunkten Xp zu berechnen.
  - **observeEventsIn(start: Date, end: Date): StateFlow<List<Event>>**
    Gibt den aktuellen Zusatnd aller Appointments und SharedEvents in einem Vorgegebenen Zeitraum als StateFlow zurück.
  - **observeToDo(): StateFlow<List<ToDo>>**
    Gibt den aktuellen Zustand aller noch nicht erledigten ToDo als StateFlow zurück.
  - **refreshEvent(EventID): StateFlow<Event>**
    Holt den aktuellen Zustand des spezifizierten Events vom Server und speichert ihn in der Lokalen Datenbank.
  - **refreshEventsIn(start: Date, end: Date): void**
    Holt den aktuellen Zustand aller Events in dem angegebenem Zeitraum vom Server und speichert ihn in der Lokalen Datenbank.
  - **refreshToDo(): void**
    Holt den aktuellen Zustand aller noch nicht Erledigter ToDo vom Server und speichert ihn in der Lokalen Datenbank.
  - **updateEvent(updatedEvent: Event): Result<Unit>**
    Ändert das Event in der datenbank mit der id des übergebenen Events zu dem übergebenen Event. Existiert kein solches Event für den user wird es hinzugrfügt.

.

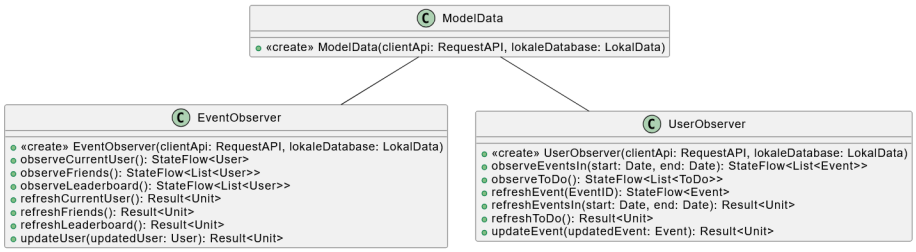


Abbildung 3.15: Klassendiagramm der ModelData Klasse und ihren Observern.

XpDistributor Die Verteilung von XP für Events wird durch den XpDistributor Realisiert. Dabei ist keine Verbindung zum Server nötig. Intern besitzt die Klasse das Attribut **user**, welches den angemeldeten Nutzer beschreibt und durch Kommunikation mit den Observern stehts aktuell ist. Zusätzlich besitzt es ein Attribut **randomNumberGenerator** der Klasse Random, um Zufallsentscheidungen für die Xp-Verteilung wie in Abschnitt 1.2.1 beschrieben durchzuführen. Die klasse besitzt die folgenden Öffentlichen Methoden:

- **calculateNextDay(): void**
  Diese funktion berechnet für jeden Appointment und jedes SharedEvent des Kommenden Tages einen Xp wert. Dabei wird das XP objekt aller Appointments in dieser Zeitspanne aktualisiert. Falls am aktuellen Tag noch garkein SharedEvent oder Appointment einen Xp wert zugeteilt worden ist wird dies in gleicherweise wie am folgetag gemacht.

- **calculateToDo(todo: ToDo): XP**
  Die Funktion bestimmt die Erfahrungspunkte, welche das erledigen des ToDo als belohnung vergibt.

Zur umsetzung soll die private funktion **calculateDay(List<Event>)** verwendet werden.

Null-Objects Wie im Pflichtenheft beschrieben existieren Null-Objects sowhol für Gruppen als auch Xp. Dabei wird für Group die Klasse NoGroup verwendet, und für XP die Klasse NoXP.

- **NoGroup**
  - **groupID: String**
    Der Indentifikator des Platzhalters ist 0
  - **groupName: String**
    Der Name von NoGroup objekten ist Keine Gruppe.
  - **groupColour: ?**
    Gibt die Farbe der Gruppe als ? zurück.

- **NoXP**
  - **XP: int**
    Das NoXP objekte gibt -1 als Wert zurück. Das View-Model erkennt NoXP und es wird nicht angezeigt.

ReportData Benötigt zur Initilaisierung ein ModelData Objekt, sowie zwei LocalDateTime objekte, welche einen Zeitraum festlegen. Anschließend stellt das Objekt folgende Methoden bereit, um Informationen über diese Zeitspanne auszulesen.

- **doneAppointments(): List¡Appointment¿**
  Liste aller abgeschlossenen Appointments in der angegebenen Zeitspanne.
- **doneToDos(): List¡ToDo¿**
  Liste aller abgeschlossenen ToDos in der angegebenen Zeitspanne.
- **completedAchievements(): List¡Achievement¿**
  Liste aller abgeschlossenen Achievements in der angegebenen Zeitspanne.

3.3.3.6 Lokale Datenspeicherung Für die lokale Datenspeicherung werden Room Entity-Klassen verwendet, die Modellierungen der bekannten Interfaces und Klassen darstellen. Dabei werden die Rückgabewerte der Interface-Methoden als Attribute gespeichert und über Getter- und Setter-Methoden zugänglich gemacht.

- **MeDB** (modelliert User)
  Beschreibt das eigene Benutzerobjekt.
- **FriendsDB** (modelliert User)
  Beinhaltet alle Freunde, die der Nutzer hinzugefügt hat.
- **LeaderboardDB** (modelliert LeaderboardEntry)
  Beinhaltet die besten Spieler und dient zur Anzeige der Rangliste.
- **AppointmentDB** (modelliert Appointment)
  Beinhaltet alle Appointments des Nutzers.
- **ToDoDB** (modelliert ToDo)
  Beinhaltet alle ToDos des Nutzers.
- **SharedEventDB** (modelliert SharedEvent)
  Beinhaltet alle Shared Events, denen der Nutzer beitreten kann oder bereits beigetreten ist.
- **AchievementDB** (modelliert Achievement)
  Beinhaltet alle Achievements.
- **EventCompletionDB** (modelliert CompletionDetail)
  Beinhaltet den Abschlussstatus von Events. Der `identifier` entspricht der `eventID` des Events und ist ein Fremdschlüssel in `AppointmentDB`, `ToDoDB` oder `SharedEventDB`. Dadurch ist die 1:1-Beziehung eindeutig definiert.
- **AchievementCompletionDB** (modelliert CompletionDetail)
  Beinhaltet den Abschlussstatus der Achievements. Der `identifier` entspricht dem Namen des Achievements und ist ein Fremdschlüssel in `AchievementDB`, wodurch ebenfalls eine eindeutige 1:1-Beziehung entsteht.

Der Zugriff auf die lokalen Tabellen wird durch drei Data-Access-Objekte (DAO) gekapselt. Diese stellen eine klare Trennung zwischen Persistenzschicht und

Domänenlogik sicher. Neben dem reinen Datenzugriff übernehmen die DAO-Klassen zusätzlich die Transformation der Datenbank-Entities in ihre entsprechenden Domänenobjekte. So wird beispielsweise ein Objekt vom Typ `AppointmentDB` in ein `Appointment`-Objekt überführt.

- **UserDao**
  Der `UserDao` ist für den Zugriff auf alle benutzerbezogenen Daten zuständig. Dazu zählen sowohl die eigenen Profildaten als auch Freundes- und Ranglisteninformationen.
  - **personalInfo(): Result<PersonalUser>**
    Liefert die lokal gespeicherten Informationen des aktuellen Nutzers.
  - **updatePersonalInfo(updatedUser: PersonalUser): void**
    Aktualisiert die gespeicherten Benutzerdaten.
  - **friends(): List<User>**
    Gibt eine Liste aller gespeicherten Freunde zurück.
  - **addFriend(username: String): Result<Unit>**
    Fügt einen neuen Freund zur lokalen Freundesliste hinzu.
  - **removeFriend(username: String): Result<Unit>**
    Entfernt einen Freund aus der Freundesliste
  - **leaderboard(): List<LeaderboardEntry>**
    Gibt die lokal gespeicherten Einträge des Leaderboards zurück.
  - **updateLeaderboard(entries: List<LeaderboardEntry>): Result<Unit>**
    Aktualisiert die gespeicherten Ranglisten-Einträge.
- **EventDao**
  Der `EventDao` verwaltet alle termin- und aufgabenbezogenen Daten. Er kapselt den Zugriff auf Appointments, ToDos und Shared Events und stellt zeitbasierte Abfragen bereit.
  - **appointmentsIn(start: LocalDateTime, end: LocalDateTime): List<Appointment>**
    Liefert alle Appointments innerhalb des angegebenen Zeitraums.
  - **toDosIn(start: LocalDateTime, end: LocalDateTime): List<ToDo>**
    Liefert alle ToDos mit einer Deadline innerhalb des angegebenen Zeitraums.

  - **insertEvent(event: Event): Result<Unit>**
    Speichert ein neues Event in der lokalen Datenbank.
  - **updateEvent(event: Event): Result<Unit>**
    Aktualisiert ein bestehendes Event.
  - **deleteEvent(event: Event): Result<Unit>**
    Entfernt ein Event aus der lokalen Datenbank.

- **AchievementDao**
  Der `AchievementDao` verwaltet alle Informationen zu Achievements und deren Abschlussstatus.
  - **completedAchievements(): List<Achievement>**
    Gibt alle abgeschlossenen Achievements zurück.
  - **allAchievements(): List<Achievement>**
    Gibt alle bekannten Achievements unabhängig vom Abschlussstatus zurück.
  - **updateAchievement(achievement: Achievement): Result<Unit>**
    Aktualisiert den Fortschritt oder Abschlussstatus eines Achievements.

Die zentrale Datenbankklasse **LocalData** wird als Singleton realisiert. Da die Anwendung zu jedem Zeitpunkt ausschließlich eine Instanz der lokalen Datenbank benötigt, wird so ein konsistenter Zugriff auf die gespeicherten Daten sichergestellt und redundante Instanzen vermieden.

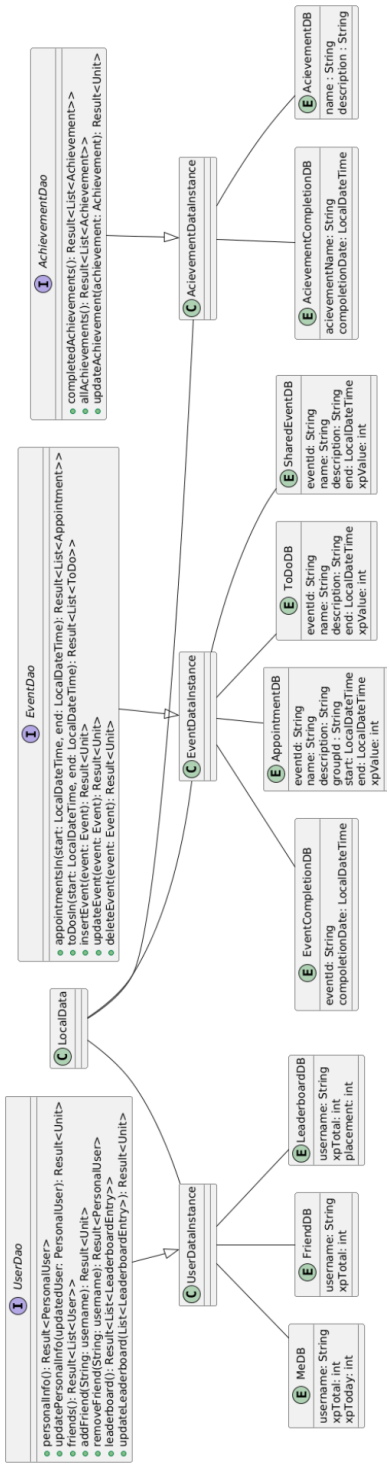


Abbildung 3.16: Klassendiagramm der Lokalen Datenbank. Zeigt welche Dao mit welchen Entitys Interagieren. Getter und settermethoden der Entitys wurden weggelassen.

A SQL-Schemadefinition

```
CREATE TABLE User (
  UserID VARCHAR(255) PRIMARY KEY,
  Name VARCHAR(255),
  ExperiencePoints INT
);

CREATE TABLE Event (
  EventID INT PRIMARY KEY,
  Name VARCHAR(255),
  Beschreibung TEXT,
  Ende TIMESTAMP,
  Erfahrungspunkte INT,
  UserID VARCHAR(255) NOT NULL,
  FOREIGN KEY (UserID) REFERENCES "User"(UserID)
);

CREATE TABLE Gruppe (
  GroupID INT PRIMARY KEY,
  Name VARCHAR(255)
);

CREATE TABLE EventCompletionDetail (
  EventID INT PRIMARY KEY,
  Completed BOOLEAN,
  CompletionDate TIMESTAMP,
  FOREIGN KEY (EventID) REFERENCES Event(EventID)
);

CREATE TABLE Appointment (
  EventID INT PRIMARY KEY,
  Start TIMESTAMP,
  GroupID INT NOT NULL,
  FOREIGN KEY (EventID) REFERENCES Event(EventID),
  FOREIGN KEY (GroupID) REFERENCES Gruppe(GroupID)
);
```

```
CREATE TABLE ToDo (
  EventID INT PRIMARY KEY,
  FOREIGN KEY (EventID) REFERENCES Event(EventID)
);

CREATE TABLE SharedEvent (
  EventID INT PRIMARY KEY,
  MinimumErfahrungspunkte INT,
  FOREIGN KEY (EventID) REFERENCES Event(EventID)
);
```